IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 22 May 2025, accepted 18 June 2025, date of publication 25 June 2025, date of current version 13 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3582892

SURVEY

Cybersecurity in DevOps Environments: A Systematic Literature Review

ROBERTO CARLOS BAUTISTA RAMOS AND SANG GUUN YOO, (Senior Member, IEEE)
Departamento de Informática y Ciencias de la Computación, Escuela Politécnica Nacional, Quito 170525, Ecuador
Corresponding author: Sang Guun Yoo (sang.yoo@epn.edu.ec)

ABSTRACT This systematic literature review provides a comprehensive analysis of the most critical cybersecurity challenges in DevOps environments. Through a rigorous examination of 62 peer-reviewed articles published between 2016 and 2025, we identified recurring threats, active attack vectors, structural vulnerabilities, mitigation strategies, and their technical impact on system performance and operational resilience. The analysis revealed that the most significant threats are related to uncontrolled automation, exposure of sensitive secrets in CI/CD pipelines, lack of mutual authentication between distributed services, supply chain attacks, and the use of unauthorized tools (Shadow IT). These threats simultaneously compromise core security principles, including integrity, confidentiality, and traceability. The most frequent attack vectors include code injection in CI/CD pipelines, unrestricted access to public repositories, remote execution via default configurations, and lateral movement in flat architectures. We identified 27 recurrent vulnerabilities throughout the DevOps lifecycle. The most critical include the absence of automated security testing, poor management of secrets, and reliance on unverified third-party components. More than 30 technical and organizational countermeasures were documented, such as SAST/DAST/IAST scans, infrastructure-as-code validation, secure credential storage via vaults, and integrated practices like DevSecOps and compliance-as-code. When properly implemented, these strategies do not degrade system performance and may even enhance resilience and stability. Nonetheless, a lack of comparative empirical validation in most reviewed studies limits the generalizability of proposed solutions. These findings establish a foundation for future research in emerging domains, such as the Internet of Things, where continuous, adaptive, and verifiable security is paramount for automated and dynamic environments.

INDEX TERMS DevOps, cybersecurity, threats, vulnerabilities, attack vectors, mitigation, systematic literature review.

I. INTRODUCTION

Current technology enables organizations to carry out digital transformation processes, which in turn significantly affects the development and operation of software systems. Within this context, the DevOps paradigm has emerged as an optimal solution to the challenge of accelerating value delivery by improving software quality as well as collaboration between Development (Dev) and Operations (Ops) teams. The principles of DevOps include automation, Continuous Integration/Continuous Delivery (CI/CD), Infrastructure as Code (IaC), continuous monitoring, and cyclic interactions, creating an adaptive system in a highly dynamic environment. However, this operational flexibility has led to increased complexity in the attack surface, which has drawn greater attention to cybersecurity issues in these highly orchestrated and automated environments [1], [2].

DevOps environments pose a challenging problem in the development of efficient and sustainable security mechanisms due to the speed at which systems are built, tested, and deployed. Traditional security, which was applied reactively or at late stages of the software lifecycle, is insufficient in environments where deployments occur multiple times per day. Therefore, security must be transformed into an integrated discipline that is incorporated from the early stages of development and maintained throughout system

The associate editor coordinating the review of this manuscript and approving it for publication was Amjad Mehmood.

operation, evolving continuously. Based on this definition, DevSecOps emerges as the new branch of DevOps that adds security as a recurring element in the software pipeline for its constant automation and application throughout the software lifecycle [3], [4].

The use of methodologies such as shift-left security allows development teams to integrate security controls from the early stages of development and greatly reduce the probability of exploiting flaws in advanced production stages. This philosophy has become highly relevant in continuous delivery environments, where the prevention of vulnerabilities in the code at early stages becomes a priority to maintain system resilience. In this way, the formal introduction of cybersecurity principles in DevOps contexts was established in 2017, when Gartner formalized the term DevSecOps in its report *Innovation Insight for DevSecOps*, providing a formally accepted framework [5].

The effective adoption of the DevSecOps approach involves combining several practices, such as static code analysis, dynamic application scanning, third-party dependency verification, secret management, policy as code, integration of access controls in automation tools, and validation of configurations in ephemeral infrastructure environments such as containers or serverless functions. These practices not only require advanced technical tools but also demand a cultural shift within the organization, where security becomes a shared responsibility rather than an isolated or subsequent role. However, this integration brings significant challenges, such as resistance to change among development teams, lack of training in secure practices, the complexity of automating controls without affecting productivity, and the need to manage a high volume of security alerts with a low false positive rate [6].

On the other hand, the arrival of new technologies such as Kubernetes, Docker, Terraform, and CI/CD platforms like Jenkins, GitLab CI/CD, GitHub Actions, or CircleCI has opened a world of possibilities for automation. However, it has also introduced new attack vectors. Security threats in DevOps are not limited to source code errors; they also include misconfigured infrastructure components, compromised supply chains, third-party libraries with known vulnerabilities, and accidental exposure of API keys, among other critical risks. In fact, recent incidents such as the SolarWinds supply chain attack and the Codecov security breach have demonstrated how malicious actors can infiltrate DevOps environments using legitimate tools, affecting not only a single organization but also multiple customers interconnected through the digital value chain [7], [8].

In this context, scientific research has produced a growing number of works addressing various facets of security in DevOps. These range from case studies on the implementation of DevSecOps practices in real organizations to proposals for maturity models, frameworks for secure integration, metrics to evaluate pipeline protection levels, and automated scanning tools for containers or source code. However, this body of knowledge is scattered, fragmented, and presents varying levels of theoretical depth and empirical validity. This study offers a systematic literature review aimed at analyzing security challenges in DevOps environments, identifying the strategies employed to mitigate threats and vulnerabilities, and examining how these strategies influence system performance. To ensure the methodological rigor of the study, the approach proposed by Kitchenham and Charters is adopted, which is widely recognized in Software Engineering and provides clear guidelines for protocol formulation, primary source identification, application of inclusion and exclusion criteria, as well as extraction and synthesis of relevant data [9].

This study provides a comprehensive view of security in DevOps environments, addressing the main threats and attack vectors, along with the documented methodological approaches to mitigate their impact on security and operational efficiency. It also examines inherent vulnerabilities that can be exploited by malicious actors, analyzing effective corrective measures to reduce risks. Finally, it explores how mitigation strategies influence system performance, identifying the most efficient ones in applied scenarios. Through a systematic review, this work aims to provide a key resource for researchers and professionals, offering technical and scientific evidence to strengthen security in modern software development.

II. METHODOLOGY

This research was carried out through a Systematic Literature Review (SLR), strictly following the methodological guidelines established by Kitchenham and Charters [9]. This methodology provides a structured and replicable framework to identify, assess, and synthesize the scientific evidence available on a specific topic, minimizing potential biases and strengthening the validity of the findings. The application of this technique is particularly relevant in rapidly evolving technological contexts such as cybersecurity in DevOps environments, where knowledge is scattered across multiple sources and disciplines.

The review process was organized into four fundamental stages, illustrated in Figure 1 and described in detail below: (1) formulation of the research questions, aimed at defining the analytical focus of the study; (2) systematic identification and selection of relevant primary studies using reproducible search strategies in high-impact scientific databases; (3) critical appraisal of the methodological quality of the selected studies through explicit inclusion, exclusion, and quality criteria; and (4) synthesis and interpretation of the results, in order to extract patterns, trends, challenges, and relevant strategies from the analyzed corpus.

This systematic approach ensures a deep, objective, and methodologically sound analysis, providing a comprehensive view of the state of the art and emerging practices related to the integration of security into DevOps development cycles.

The main objective of this review was to collect, filter, and synthesize primary studies relevant to the field

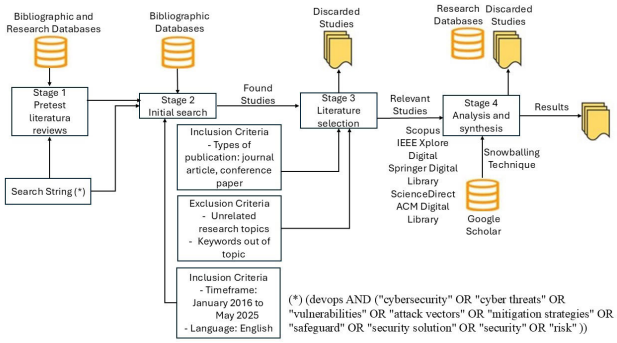

FIGURE 1. Stages of the systematic literature review process.

of cybersecurity in DevOps environments, considering as primary study any scientific publication that reports original findings, empirical results, or theoretical contributions based on evidence directly obtained by the authors. The SLR, as a consolidated technique within the evidence-based research paradigm, is designed to answer specific research questions through a rigorous, transparent, and reproducible process, based on the prior definition of a structured review protocol. This protocol encompasses aspects such as the design of search strategies, definition of Boolean strings, selection of academic sources, and cross-validation of the results obtained, thus ensuring the traceability and reliability of the entire research process.

A. PHASE 1: INITIAL SEARCH

This phase focused on the methodological and operational planning of the systematic literature review through the structuring of a rigorous and reproducible research protocol. The process was divided into two sub-stages: (a) protocol design and (b) protocol execution.

a) Research Protocol Design

A set of Research Questions (RQs) was defined to explore in a structured manner the challenges, strategies, practices, and emerging trends surrounding security in DevOps environments. The research questions, along with their motivations, are detailed in Table 1. In parallel, a structured search string was constructed (Table 2) using Boolean operators, controlled vocabulary, and relevant synonyms, with the aim of maximizing both recall and precision in the retrieval of pertinent studies.

The selection of bibliographic sources focused on highly relevant databases in the field of software engineering and cybersecurity, including: Scopus, IEEE Xplore Digital Library, SpringerLink, ScienceDirect, and ACM Digital Library. The time frame for analysis was set from 2016 to 2025, in order to capture the contemporary evolution of the DevSecOps approach, framed by two key milestones:

- In 2016, Julien Vehent published the book *Securing DevOps*, a pioneering work that laid the technical foundations for the systematic incorporation of security practices into automated development pipelines [10].
- In 2017, Gartner formally included the concept of DevSecOps in its report *Innovation Insight for DevSecOps*, recognizing it as a strategic trend in modern risk management [5].

Additionally, the protocol established a language criterion requiring that only publications written in English be included, in order to ensure an international, technical, and specialized focus aligned with the standards of high-impact scientific literature.

b) Protocol Execution

Once the protocol was defined, systematic searches were conducted in the selected databases, applying the predefined search string in the fields of title, abstract, and keywords. The result of this phase was the construction of a preliminary document corpus composed of scientific publications directly related to the subject of study: cybersecurity in DevOps environments. This initial set of studies was subsequently subjected to filtering, quality assessment, and data extraction processes, corresponding to the subsequent stages of the methodological framework.

B. PHASE 2: BIBLIOGRAPHY SELECTION

This stage included the following activity: **a)** Conducting a detailed analysis of the titles, abstracts, and keywords of the identified studies, applying inclusion and exclusion criteria. The inclusion criteria were defined as the exclusive acceptance of articles published in peer-reviewed journals and papers presented at relevant academic conferences. On the other hand, the exclusion criteria consisted of discarding studies whose research approaches were not related to the main topic or that included keywords outside the area of interest.''

C. PHASE 3: ANALYSIS AND SYNTHESIS

This stage was structured as follows: **a)** The full content of the studies was accessed through highly regarded academic databases, such as Scopus, IEEE Xplore Digital Library, Springer Digital Library, ScienceDirect, and ACM Digital Library. These sources were selected due to their relevance in publishing high-quality scientific research and their coverage in areas related to technology, software development, and cybersecurity. Additionally, they ensure access to peer-reviewed articles and a wide range of specialized resources, thereby guaranteeing the scientific validity of the document corpus. **b)** The complete content of the studies was assessed based on predefined inclusion criteria. Only studies whose focus was semantically related to the DevOps paradigm and that addressed specific aspects of cybersecurity, threats, vulnerabilities, and mitigation strategies were considered relevant. **c)** To enrich the final document corpus, the snowballing technique was applied. This process involved using Google Scholar (GS) to identify studies that cited the initially selected documents, as well as conducting an exhaustive review of their references, following the methodology described by [11]. This technique allowed for

TABLE 1. Research questions and motivations.

Research Question	Motivation
RQ1. What are the main cyber threats and attack vectors impacting DevOps environments, and which methodological approaches have been documented to mitigate their impact on system security and operational efficiency?	The motivation behind this question lies in the need for a comprehensive characterization of cyber threats and attack vectors affecting DevOps environments, given their highly automated, ephemeral architecture geared toward continuous integration and delivery. The goal is to systematically analyze adversarial mechanisms that exploit attack surfaces introduced by CI/CD pipelines, containers, infrastructure as code, and orchestration tools, as well as to compile effective methodological approaches.
RQ2. What are the main inherent vulnerabilities in DevOps environments that facilitate exploitation by malicious actors, and what corrective measures have been implemented to mitigate their impact on system security?	This question arises from the need to identify systemic, technical, and procedural vulnerabilities inherent to DevOps practices which, due to their focus on agility and automation, tend to introduce non-trivial exposure conditions. The aim is to establish a taxonomy of exploitable weaknesses, including insecure configurations, secret management failures, and dependency on vulnerable software components, as well as to examine empirically validated corrective measures, such as the integration of automated controls, security as code, and early verification mechanisms via shift-left approaches.
RQ3. How do threat and attack mitigation strategies in DevOps environments influence system performance, and which have proven to be the most effective in applied scenarios?	This question addresses the inherent tension between implementing cybersecurity measures and maintaining optimal levels of performance, scalability, and operational agility in DevOps pipelines. It seeks to examine, based on empirical evidence and applied studies, the quantitative and qualitative impact of mitigation strategies on productivity, development throughput, deployment times, and environment latency. It also aims to identify practices that, due to their effectiveness and low computational cost, have been validated as optimal trade-offs between protection and efficiency.

TABLE 2. Search strings applied in each database.

Database	Search String	Articles Found
Scopus	TITLE-ABS-KEY ((devops AND ("cybersecurity" OR "cyber threats" OR "vulnerabilities" OR "attack vectors" OR "mitigation strategies" OR "safeguard" OR "security solution" OR "security" OR "risk"))) AND PUBYEAR > 2015 AND PUBYEAR < 2026 AND (LIMIT-TO (DOCTYPE, "ar")) AND (LIMIT-TO (LANGUAGE, "English"))	131
IEEE Xplore Digital Library	(devops AND ("cybersecurity" OR "cyber threats" OR "vulnerabilities" OR "attack vectors" OR "mitigation strategies" OR "safeguard" OR "security solution" OR "security" OR "risk"))	25
Springer Digital Library	(devops AND ("cybersecurity" OR "cyber threats" OR "vulnerabilities" OR "attack vectors" OR "mitigation strategies" OR "safeguard" OR "security solution" OR "security" OR "risk"))	479
ScienceDirect	(devops AND ("cybersecurity" OR "cyber threats" OR "vulnerabilities" OR "attack vectors" OR "mitigation strategies" OR "safeguard" OR "security solution" OR "security" OR "risk"))	83
ACM Digital Library	[All: devops] AND [[All: "cybersecurity"] OR [All: "cyber threats"] OR [All: "vulnerabilities"] OR [All: "attack vectors"] OR [All: "mitigation strategies"] OR [All: "safeguard"] OR [All: "security solution"] OR [All: "security"] OR [All: "risk"]] AND [E-Publication Date: (01/01/2016 TO 12/31/2025)]	158

the identification of relevant connections and expanded the analytical scope. **d)** A critical and thorough analysis was carried out both within each selected study and across the studies in the final corpus, with the aim of identifying recurring patterns, knowledge gaps, and complementary approaches. **e)** Finally, the research questions stated in the protocol were addressed by integrating findings obtained during the analysis and synthesis stage, ensuring a rigorous and evidence-based approach.

III. RESULTS AND ANALYSIS

This section presents the results obtained throughout the search process, including a detailed description of the final composition of the document corpus. This corpus consists of 62 relevant studies that were selected and classified according to predefined criteria, covering various topics and methodologies. The key findings that emerged from the reviewed studies are outlined below, along with prevailing patterns, trends, and research areas, with a particular focus on security practices and tools applied in DevOps environments.

A. INITIAL SEARCH

Searches were conducted in the bibliographic databases specified in the research protocol, applying the search string defined in Table 2. This process resulted in the identification of a total of 876 potentially relevant studies for the objectives of this research.

B. BIBLIOGRAPHIC SELECTION

In Stage 3, 657 out of the 876 studies analyzed were filtered out. These were excluded due to duplication across sources, lack of thematic alignment, absence of explicit references to DevOps environments in titles or abstracts, and the use of non-pertinent keywords. As a result, 219 articles were selected for further consideration.

C. ANALYSIS AND SYNTHESIS

To ensure the validity and reliability of the study selection process, expert judgment was applied as a validation mechanism for the inclusion and exclusion criteria. For this purpose, a panel was formed, consisting of two researchers with proven expertise in software engineering, agile methodologies,

and specifically DevOps practices. This group critically reviewed the defined criteria, ensuring their alignment with the research questions (RQ) and their suitability within the technical and scientific context of the DevOps domain. The feedback obtained allowed for the refinement of the criteria and ensured that the selected studies were relevant, up-to-date, and directly related to the topics addressed in this systematic review. To guarantee the validity and reliability of the selected studies on DevOps, quality criteria were applied, resulting in the acceptance of 49 studies Subsequently, the snowballing technique was applied following the methodology described by Jalali and Wohlin [11], with the objective of exhaustively expanding the corpus coverage. In this phase, Google Scholar (GS) was used as a complementary source, enabling the identification of 13 additional studies that met the established inclusion criteria. As a result, a final corpus comprising 62 primary studies was consolidated, providing a solid and comprehensive foundation for the proposed systematic analysis, as presented in Figure 2.

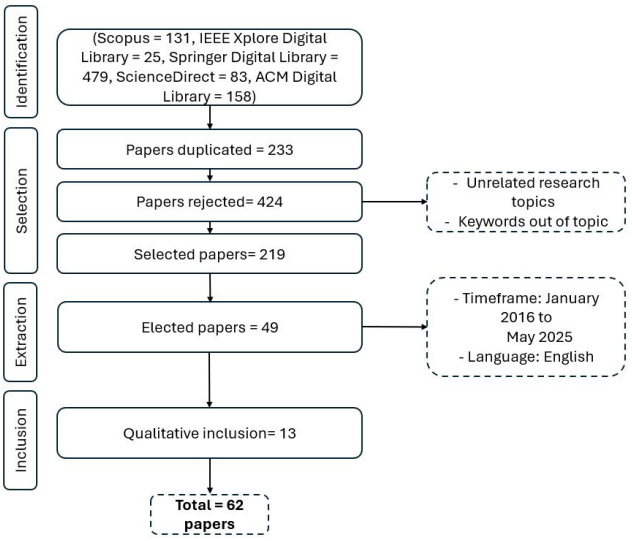

FIGURE 2. Flow of article identification and selection from databases and other sources.

D. CYBERSECURITY LANDSCAPE IN DEVOPS: LITERATURE FINDINGS

This section presents a structured synthesis of the findings derived from the systematic literature review, which focused on the rigorous examination of cyber threats, attack vectors, structural vulnerabilities, mitigation strategies, and their impact on operational efficiency in DevOps environments. Through thematic and comparative analysis of the included studies, recurrent patterns, consolidated methodological approaches, and significant gaps were identified, enabling a more precise delineation of the current state of cybersecurity in contexts of automation and continuous delivery. One notable finding is that over 80% of the studies explicitly focus on issues related to threats and attack vectors that compromise the integrity of CI/CD pipelines, containerized environments, infrastructure as code (IaC), and automation tools. This thematic concentration reveals a high sensitivity within the scientific literature regarding emerging attack surfaces stemming from agile development practices and dynamic orchestration. A growing concern is also identified regarding the inherent weaknesses of automated infrastructure, particularly with respect to the lack of integrated security controls from the early stages of the software development lifecycle. In response to these deficiencies, multiple studies document mitigation strategies aligned with the principles of DevSecOps, such as the use of automated static and dynamic analysis tools, implementation of policy-as-code mechanisms, secure management of secrets and credentials, and the adoption of continuous control frameworks and automated compliance mechanisms. In this regard, some works explore how these security practices affect system agility, performance, and operational resilience, providing empirical evidence on the necessary balance between defensive robustness and delivery efficiency. The results have been organized into three key dimensions, corresponding to the research questions posed in this study: (a) Cyber threats in DevOps environments, (b) Attack vectors in DevOps environments, (c) Methodological approaches for threat mitigation in DevOps environments, (d) Vulnerabilities in DevOps environments, (e) Corrective measures to mitigate vulnerabilities, (f) Effects and implications of security mitigation strategies on the performance of DevOps environments, and (g) Correlation between threats, vulnerabilities, associated risk, and their implications for the security triad.

1) THREATS IN DEVOPS ENVIRONMENTS

Threats in DevOps environments refer to potential events or conditions that may be exploited by malicious actors to compromise the security, integrity, or availability of systems during the continuous software development and delivery cycle. These threats arise from latent vulnerabilities in practices, configurations, tools, and automated workflows that are inherent to the DevOps culture.

The results of the analysis reveal a significant set of recurrent and critical threats, notably including the delayed integration of security measures [25], [27], [30], [33], misconfiguration of Infrastructure as Code (IaC) and containers [14], [17], [26], [28], [31], [47], and poor management of privileged access [13], [21], [45], [50], [55], which enable unauthorized escalation scenarios. Additional threats include data leakage due to unencrypted or improperly managed storage [39], [41], [44], [46], compromised software supply chains [20], [35], [44], [51], [57], and the use of unauthorized tools (Shadow IT) [14], [22], [36], [47], [60], introducing unaudited external vectors.

Emerging threats also include the reliance on legacy scripts without security validation [26], [35], [58], the lack of authentication between microservices or integrated tools [12], [13], [48], [61], and false positives or negatives in automated scanning tools, which reduce the accuracy

TABLE 3. Cybersecurity threats in DevOps environments.

Threat	Type of Exploited Vulnerability	Importance (Details)	Frequency (Details)	References
Delayed security integration	Late implementation of security controls	High: allows vulnerabilities to reach production	High: common practice in immature DevOps models	[25] [27] [30] [33]
IaC and container misconfigurations	Incorrect or default configurations	High: may expose critical services	High: frequent in deployments without validation	[14] [17] [26] [28] [31] [47]
Poorly managed privileged access	Uncontrolled privilege escalation and access	High: compromises internal systems and critical resources	Medium-High: common in CI/CD pipelines and deployments	[13] [21] [45] [50] [55]
Vulnerable third-party dependencies	Insecure or unmaintained libraries	High: gateway for supply chain attacks	High: widely documented in CI/CD analyses	[20] [28] [44] [57] [60]
Human errors and manual practices	Lack of expertise or poor practices	Medium-High: introduces insecure configurations	High: common in teams with limited security training	[22] [30] [36] [47] [51]
Insider threats	Excessive permissions and lack of auditing	High: enables sabotage or internal data theft	Medium: in organizations without strong controls	[22] [48] [54]
Anomalies and unauthorized activities	Absence of continuous monitoring and analysis	High: can conceal real-time attacks	Medium: depends on monitoring automation	[25] [41] [59]
CI/CD pipeline compromise	Uncontrolled modifications and insecure scripts	High: enables malicious code execution	Medium-High: in pipelines with weak controls	[13] [17] [26] [28] [33]
Data leakage via unencrypted storage	Lack of encryption at rest and in transit	High: exposes sensitive information	Medium: depends on applied policies	[39] [41] [44] [46] [50]
Supply chain attacks	Compromised dependencies or tools	High: injects malware into the DevOps cycle	High: critical threat in CI/CD	[20] [35] [44] [51] [57]
Injection via deployment scripts	Weak validations and shell injection	High: enables remote command execution	Medium: common in loosely controlled processes	[28] [36] [42] [58]
Lack of authentication between microservices	Open or unauthenticated connections	High: enables unauthorized access	Medium-High: frequent in distributed architectures	[12] [13] [48] [61]
Misconfigured static analysis	False negatives due to poor rule calibration	Medium: false sense of security	Medium: depends on team expertise	[38] [42] [46] [52]
Unauthorized tools (Shadow IT)	Lack of governance and control	Medium-High: introduces unevaluated software	Medium: common without centralized policy	[14] [22] [36] [47] [60]
Mismanaged public repositories	Code or secret exposure due to human error	High: reveals sensitive configurations	High: frequent without permission controls	[13] [20] [30] [33] [51]
Internal network policy errors	Lack of segmentation and traffic control	High: allows lateral movements	Medium: depends on orchestration and visibility	[21] [23] [54]
Lack of version control in container images	Use of outdated or compromised images	High: retains known vulnerabilities	High: in fast-paced deployments without maintenance	[13] [17] [20] [29] [33]
Legacy insecure scripts	Reuse of unaudited code	Medium-High: logical errors or insecure settings	Medium: common in technical debt	[26] [28] [35] [45]
False positives/negatives in scanning tools	Misinterpretation of findings during analysis	Medium: affects decisions and leaves threats untreated	Medium: depends on tool calibration	[38] [42] [49] [52]
Authentication failures in integrated tools	Implicit trust without authentication	High: enables lateral access between modules	Medium-High: in multi-vendor architectures	[12] [13] [30] [32] [48]

in detecting actual vulnerabilities [38], [42], [49], [52]. These threats, distributed throughout the DevOps pipeline, underscore the need to strengthen security controls and adopt a proactive and continuous stance against cyber risk.

Table 3 provides a summary of the threats identified in DevOps environments. The table is structured as follows: from left to right, it includes the type of threat, the type of vulnerability exploited, the importance, frequency, whether it is common, and the scientific references where each is mentioned.

The assessment of threats in DevOps follows NIST SP 800-30 [74], CVSS v3.1 [75], and ISO/IEC 27005 [76]. High-importance threats compromise system confidentiality, integrity, and availability (CIA) and require immediate controls due to their high occurrence probability [74], [77]. Medium-importance threats have a localized impact and depend on specific conditions for exploitation, while low-importance threats are rare, low-impact, and manageable with minimal intervention [74], [75], [76].

2) ATTACK VECTORS IN DEVOPS ENVIRONMENTS

Attack vectors in DevOps environments represent the specific mechanisms through which malicious actors can exploit vulnerabilities and materialize threats within the continuous cycle of development, integration, and operations. The analysis shows that these vectors are closely linked to insecure configurations, mismanaged automation, exposed credentials, and lack of segmentation between services.

One of the most critical vectors is code injection into CI/CD scripts, enabling the automated execution of

malicious commands within continuous integration and deployment environments [13], [17], [26], [28], [33]. Another high-impact vector involves uncontrolled access to public repositories, allowing for the exfiltration of secrets, tokens, or exposed configurations [13], [20], [30], [33], [51]. Remote code execution is also documented through default settings in containers and Infrastructure as Code (IaC), which often enable unauthenticated or exposed services [14], [17], [26], [31], [47], [56].

Privilege escalation via hardcoded or poorly managed credentials is another common vector [13], [21], [45], [50], [61], as is the injection of malicious dependencies during the software build process—one of the main entry points in supply chain attacks [20], [28], [35], [44], [60].

An often underestimated vector is the reuse of insecure legacy scripts, which may contain outdated commands, misconfigurations, or excessive permissions. These scripts frequently remain in place due to delivery pressures and lack of technical governance. As noted in [37], intensive use of microservices without a controlled architecture can increase technical debt, thereby expanding the attack surface by leaving insecure components in production.

Finally, additional vectors were identified, including the use of unauthorized tools (Shadow IT) [22], [36], [47], [60], the integration of systems without cross-authentication [12], [30], [32], [48], and the absence of network policies or segmentation, which facilitates lateral movements within orchestrated architectures [21], [23], [54], [61]. These vectors, aligned with the threats previously described, provide a clearer understanding of the most exploited entry points in modern DevOps environments. In this context, emerging technologies such as deep learning have proven effective in detecting complex threats. For example, [53] presents a review of deep neural network-based architectures for protecting wireless systems, which could be extrapolated to distributed DevOps environments, where predictive models enable the detection of anomalies that traditional rule-based systems may overlook.

Table 4 summarizes the most relevant attack vectors in DevOps environments. The table is structured as follows: from left to right, attack vector type, associated threat, importance, frequency and the scientific references where each is mentioned. Critical attack vectors enable direct access or manipulation of essential assets via exploitable routes such as repository exposure or CI/CD pipeline tampering, frequently identified in audits and security frameworks [74], [75], [76] Mediumimpact vectors target non-essential services and require specific conditions for exploitation, while low-impact vectors depend on highly uncommon technical setups, making them infrequent in real-world audits [74], [75], [76].

3) METHODOLOGICAL APPROACHES FOR THREAT MITIGATION IN DEVOPS ENVIRONMENTS

Methodological approaches for mitigating threats in DevOps environments focus on integrating proactive and adaptive defense mechanisms into the continuous development, integration, and operations cycle.

The analysis reveals that these methodologies respond directly to specific combinations of threats and attack vectors, employing techniques ranging from automated controls to organizational policies. Notably, the implementation of digital signatures and script validation in CI/CD pipelines helps prevent code injection during the integration process [13], [17], [26], [28], [33]. Similarly, automating the scanning of Infrastructure as Code (IaC) and containers proves to be an effective strategy for mitigating misconfigurations and unintentional exposures [14], [17], [26], [31], [47], [56]. Centralized secret management and Role-Based Access Control (RBAC) emerge as critical practices to prevent privilege escalation and unauthorized access [13], [21], [45], [50], [61].

Regarding the software supply chain, the use of Software Composition Analysis (SCA) tools and secure versioning policies is effective in blocking the injection of malicious dependencies [20], [28], [35], [44], [60]. Other documented approaches relate to observability and operational security feedback, through the deployment of integrated alerts, event dashboards, and automated response systems [25], [41], [59]. In this context, [69] propose a deep learning-based approach to detect anomalous behaviors in DevOps pipelines. Their model enables real-time identification of deviations, even when threats do not match predefined patterns, enhancing response capabilities and reducing exposure time to complex attacks.

These approaches, aligned with well-established standards and frameworks such as NIST, OWASP, and DevSecOps, provide a solid foundation for strengthening the resilience and comprehensive security of modern DevOps implementations. Among these frameworks, [64] recommend the use of the OSCAL language, developed by NIST, as an effective solution to automate regulatory compliance. This approach enables the structured representation of security controls, facilitating their integration into DevSecOps pipelines and continuous verification against frameworks such as NIST SP 800-53 or ISO/IEC 27001.

Table 5 provides a summary of the methodological approaches for threat mitigation in DevOps environments. The table is structured as follows: from left to right, it presents the threat type, attack vector, methodological mitigation approach, application level, and references showing the scientific articles in which they are discussed.

4) VULNERABILITIES IN DEVOPS ENVIRONMENTS

Vulnerabilities in DevOps environments represent critical exposure points that compromise the integrity, confidentiality, and availability of systems throughout the entire development and operations lifecycle. These weaknesses arise from insecure automated practices, misconfigurations, organizational failures, and the late integration of security into continuous delivery processes.

TABLE 4. Attack vectors in DevOps environments.

Attack Vector	Associated Threat	Importance (details)	Frequency (details)	References
Code injection in CI/CD scripts	CI/CD pipeline compromise	High: automated execution of malicious code	Medium-High: in pipelines lacking validation and control	[13][17][26][28][33]
Uncontrolled direct access to public repositories	Poorly managed public repositories	High: exposure of code and secrets	High: lacking access and visibility policies	[13][20][30][33][51]
Remote execution due to default configurations	Misconfigurations in IaC and containers	High: exposes services without authentication	High: frequent without manual review or scanning	[14][17][26][31][47][56]
Privilege escalation via embedded credentials	Poorly managed privileged access	High: compromises internal resources with elevated permissions	Medium-High: in deployments lacking privilege separation	[13][21][45][50][61]
Insertion of malicious dependencies in the build	Vulnerable third-party dependencies	High: modifies behavior without detection	High: no external component scanning	[20][28][35][44][60]
Data leakage due to plaintext storage or misconfigured buckets	Data breach from unencrypted storage	High: exposes information without elevated privileges	Medium: lacking clear encryption policies	[39][41][44][46][49][50]
Abuse of unauthorized tools by staff	Use of unauthorized tools (Shadow IT)	Medium-High: introduces unaudited software	Medium: low control over devices	[22][36][47][60]
Insecure connections without mutual authentication	Lack of authentication between microservices	High: lateral movements within orchestration	Medium-High: no mTLS or JWT	[21][23][30][54][61]
Legacy scripts with unsafe commands	Dependence on insecure legacy scripts	Medium-High: destructive operations without validation	Medium: in projects with high technical debt	[26][28][35][45][37]
Unlogged manual actions	Human errors and manual practices	Medium-High: modifications without traceability	High: absence of automated controls	[22][30][36][47][51]
Exposure of S3 buckets or volumes without encryption	Data breach from unencrypted storage	High: access to sensitive data without authentication	Medium: depends on security policies	[39][41][44][46][49][50]
Lateral movement via unsegmented services	Flaws in internal network policies	High: facilitates attack propagation	Medium: lacking network segmentation	[21][23][54][61]
Command execution without validation from pipelines	Injection via deployment scripts	High: persistent remote execution	Medium: uncontrolled legacy scripts	[28][36][42][58]
Compromised software through supply chain	Software supply chain attacks	High: indirect malicious code	High: common in DevOps pipelines	[20][26][35][44][57][60]
Manipulation of environment variables	False positives/negatives in scanning tools	Medium: affects vulnerability diagnostics	Medium: depends on tool quality	[38][42][46][52]
Use of outdated or unsigned container images	Lack of version control over container images	High: exposes systems to vulnerabilities	High: common without automated scanning	[13][17][20][29][33]
Lack of feedback on anomalies	Anomalies and unauthorized activities in production	High: undetected attacks	Medium: depends on observability	[25][41][59]
Integration without cross-authentication	Authentication failures between integrated tools	High: unauthorized access among components	Medium-High: integration lacking authentication	[12][30][32][48]
Improper access due to excessive privileges	Insider threats	High: users with malicious permissions	Medium: no RBAC or audit controls	[22][48][54]
Automation without regulatory compliance	Automation misaligned with regulations	Medium-High: legal noncompliance or breaches	Medium: depends on regulatory framework	[39][47][48][50][51]

Among the most relevant vulnerabilities is the lack of automated security testing, which allows undetected flaws to propagate into production environments [5], [13], [17], [26], [30]; the exposure of secrets in CI/CD pipelines, enabling unauthorized access to critical resources [13], [14], [45], [50]; and insecure configurations in containers and Infrastructure as Code (IaC), which expand the attack surface and create favorable conditions for exploitation [14], [17], [26], [28], [31], [47].

In addition, the absence of threat modeling, lack of continuous monitoring, and the use of static analysis tools with low coverage or limited precision reduce the effectiveness of threat detection and response [25], [38], [41], [46], [52]. Other structural vulnerabilities include the use of unverified external dependencies, which compromise the software supply chain [20], [28], [35], [44], [60], and poor management of privileged access, enabling potential internal escalations [13], [21], [45], [50], [61]. These weaknesses are distributed across development, integration, deployment, and operations phases, highlighting the need for comprehensive and preventive approaches that address the dynamic and automated nature of DevOps environments.

Beyond technical deficiencies, the human dimension is a critical factor in DevOps security. The lack of a security-oriented organizational culture, pressure to release code rapidly, and insufficient specialized training can lead to costly errors and unintentional failures. In this context, [62] conducted an empirical study that identifies multiple human-induced risks within DevOps teams, demonstrating that human decision-making and poor knowledge management directly influence the emergence of vulnerabilities and compliance failures.

Table 6 provides a summary of vulnerabilities identified in DevOps environments. The table is structured as follows:

TABLE 5. Methodological approaches for threat mitigation in DevOps environments.

Threat	Attack Vector	Mitigation Methodology	Application Level	References
CI/CD pipeline compromise	Code injection in CI/CD scripts	Digital signatures integration, script validation, and isolated execution environments	CI/CD - Integration and Deployment	[13][17][26][28][33]
Poorly managed public repositories	Uncontrolled access to public repositories	Access policies enforcement, periodic audits, and repository visibility control	Development and Code Control	[13][20][30][33][51]
IaC and container misconfigurations	Remote execution via default configurations	IaC validation automation, image scanning, and tools like kube-bench	Infrastructure and Deployment	[14][17][26][31][47][56]
Poor privileged access management	Privilege escalation via embedded credentials	RBAC enforcement, MFA, credential auditing, and centralized secrets	Identity Operations and Security	[13][21][45][50][61]
Vulnerable third-party dependencies	Malicious dependencies in build process	Use of SCA tools and secure minimum version policies	Build and Dependency Management	[20][28][35][44][60]
Unencrypted storage	Data leakage via plaintext or misconfigured buckets	Encryption in transit and at rest, key management, and RBAC-based access policies	Infrastructure and Data Security	[39][41][44][46][49][50]
Missing microservice authentication	Insecure microservice connections without mutual auth	mTLS implementation, JWT-based auth, and secure API gateways	Deployment and Service Communication	[21][23][30][54][61]
Unauthorized tool usage	Abuse of unauthorized tools by internal staff	Approved tools catalog, software usage monitoring	Tool Governance and Management	[22][36][47][60]
Deployment script injection	Command execution without validation from pipelines	Script audit, secure interpreters, input validation, shell injection controls	Integration, QA and Deployment	[28][36][42][58]
Software supply chain attacks	Compromised software via supply chain	SLSA controls, third-party code scanning, provenance verification, continuous review	Build and Software Security	[20][26][35][44][57][60]
Human error and manual practices	Unlogged manual operator actions	Automation of critical tasks, dual validation, secure ops training	Operations and Personnel Security	[22][30][36][47][51]
Insecure legacy dependencies	Legacy scripts with unsafe commands	Script refactoring, internal dependency review, security linters	Development and QA	[26][28][35][45]
Inaccurate scan tools	Manipulation of environment variables in unsafe contexts	Continuous evaluation of tool accuracy, combined static, dynamic, and interactive analysis	QA and Application Security	[38][42][46][52]
Poor container image control	Use of outdated/unsigned container images	Secure versioning policies, private registries, automated image scanning	Build and Containers	[13][17][20][29][33]
Unauthorized production anomalies	Lack of feedback on operational anomalies	Observability, security event correlation, active alerts with auto-response	Operations and Monitoring	[25][41][59]
Tool integration auth failures	Tool integration without cross-authentication	Mutual authentication, token control, federated identity management	Integration and Tool Security	[12][30][32][48]
Insider threats	Improper access due to excessive privileges	Zero Trust, permission segmentation, behavioral monitoring, continuous audit	Internal Security and Access Control	[22][48][54]
Non-compliant automation	Automation without regulatory compliance	Automated control mapping (NIST, ISO, OWASP), compliance tools like OpenSCAP	Governance and Compliance	[39][47][48][50][51][64]
Lack of feedback on operational anomalies	Lack of feedback on operational anomalies	SIEM deployment, integrated alerts, and automatic feedback to development teams	Operations and Continuous Monitoring	[25][41][59]

from left to right, it includes the vulnerability, vulnerability type, exploitability, importance, frequency, whether it is common, and finally, the references indicating the scientific articles in which they are mentioned.

A vulnerability is critical if it enables remote execution, privilege escalation, or unauthorized access, with CVSS scores $\geq$ 7.0 [75] and a high occurrence probability in security assessments [74], [76]. Medium-level vulnerabilities impact secondary assets, requiring authenticated access or specific configurations, while low-level vulnerabilities have minimal security implications and require complex conditions for exploitation [74], [75], [76].

5) CORRECTIVE MEASURES TO MITIGATE VULNERABILITIES

Corrective measures aimed at mitigating vulnerabilities in DevOps environments represent a fundamental response to the security deficiencies identified across the various stages of the software lifecycle. Findings reveal a multifactorial approach in the design and implementation of these actions, ranging from automated technical strategies to governance practices and regulatory compliance. Key measures include the integration of automated security testing into CI/CD pipelines, enabling early detection of flaws prior to deployment [13], [17], [26], [30]; secure secret management through vaults, access segmentation, and periodic credential rotation [13], [14], [45], [50]; and the adoption of the DevSecOps approach, which embeds security structurally into development from its initial phases [12], [21], [24], [27].

Other critical actions involve the continuous validation of configurations in Infrastructure as Code (IaC) and containers, mitigating errors derived from unaudited automation [14], [17], [26], [28], [31], as well as active

TABLE 6. Vulnerabilities in DevOps environments.

Vulnerability	Vulnerability Type	Exploitable?	Importance (details)	Frequency (details)	References
Lack of automated security testing	Insufficient testing	Yes	High: undetected gaps may compromise the system	High: frequent in DevOps without integrated security	[13][17][26][30]
Exposure of confidential secrets	Secret management	Yes	High: may compromise credentials and access tokens	High: frequent in misconfigured pipelines	[13][14][45][50]
Misconfiguration in containers	Configuration	Yes	High: expands attack surface	High: common in microservice environments	[14][17][26][28][31][47]
Late security integration	Process	Yes	High: allows vulnerabilities into production	High: common in traditional DevOps	[25][27][30][33]
Lack of granular access control	Unrestricted access	Yes	High: unauthorized access to critical resources	Medium: depends on policy implementation	[13][21][45][50][61]
Use of vulnerable dependencies	External dependencies	Yes	High: compromised libraries may introduce attacks	High: common in projects with many packages	[20][28][35][44][60]
Compromised CI/CD pipelines	Supply chain	Yes	High: may inject malicious code into production	Medium-High: depends on control in tools like Jenkins	[13][17][26][28][33]
Poor security culture	Cultural	Indirectly	Medium-High: limits adoption of best practices	High: frequent in non-DevSecOps organizations	[22][24][27][62]
Lack of secure coding standards	Development practices	Yes	High: introduces avoidable logic and validation flaws	High: common in fast or unreviewed development	[13][14][20]
Inadequate secret management in CI/CD	Secret management	Yes	High: allows unauthorized access to critical environments	High: common in pipelines lacking dedicated tools	[13][14][20][45][50]
Lack of threat modeling	Security analysis	Yes	High: unidentified threats may be directly exploited	Medium: frequent in low-maturity environments	[25][38][46]
Exposed attack surfaces	Insecure architecture	Yes	High: enables attackers to find and exploit open services	Medium: depends on architecture design	[17][26][28][31]
Lack of security integration in DevOps tools	Tools	Yes	High: tools without controls facilitate attacks	High: frequent in immature DevSecOps setups	[12][13][21][30]
Reactive security testing	Testing process	Yes	Medium-High: allows flaws into production	Medium: depends on QA strategy adopted	[13][22][27][30]
Lack of code change traceability	Configuration management	Yes	Medium-High: hinders root cause identification for vulnerabilities	Medium: depends on versioning and audit systems	[14][19][30]
Microservices architecture complexity	Distributed architecture	Yes	High: each microservice needs individual security	High: common in modern DevOps adoption	[13][17][20]
Outdated container images	Obsolescence	Yes	High: old components contain known vulnerabilities	High: common in multi-integration environments	[13][17][20][29][33]
Unverified toolchains	Supply chain	Yes	High: public components may contain backdoors	Medium-High: frequent without scanning integrations	[20][28][29]
Poor management of privileged access	Identity management	Yes	High: enables internal attacks or privilege escalation	Medium: depends on RBAC and audit policies	[13][21][45][50][61]
Lack of security feedback from operations	Feedback cycle	Indirectly	Medium: recurring errors due to poor communication	Medium: seen in traditional structures	[25][41][59]
Automation without security validation	Automation	Yes	High: insecure flows rapidly propagate vulnerabilities	High: very common without DevSecOps practices	[13][14][25][30]
Lack of real-time cybersecurity monitoring	Monitoring and auditing	Yes	High: prevents detection of ongoing or anomalous attacks	Medium: common in under-supervised environments	[38][41][46][52]
Persistent threats in CI/CD without traceability	Supply chain	Yes	High: hides malicious code in production environments	Medium-High: seen in pipelines without historical scanning	[35][36][40][42][45]
Data leakage due to insecure storage	Data protection	Yes	High: exposes critical information like passwords or policies	High: common in shared or unencrypted storage	[39][41][44][46][49]
Security automation misaligned with regulations	Compliance	Indirectly	Medium: may result in sanctions or legal gaps	Medium: frequent when regulation is not built-in	[39][47][48][50][51]
Lack of security in static analysis tools	Tools	Yes	Medium-High: poor results may hide threats	Medium: depends on outdated tool usage	[38][42][46][52]
Excessive permissions in orchestrated services	Orchestration and access control	Yes	High: facilitates lateral movement and internal intrusions	High: common when least privilege is not enforced	[43][50][54][55][56]
Use of unverified images in CI	Supply chain	Yes	High: may allow external code execution	High: frequent when using public repositories	[44][51][57][58]

operations monitoring integrated with feedback loops to development teams, enabling agile responses to emerging incidents [25], [34], [41], [59].

Beyond early-stage adoption, it is essential to implement a coordinated set of tools and practices tailored to each phase of the DevSecOps cycle. In [71], a comprehensive review of existing solutions is presented, categorizing tools by purpose: static analysis (SAST), dynamic analysis (DAST), secure secret management, compliance-as-code, and operational monitoring. Their study provides an integrated framework that allows organizations to select and implement solutions based on their maturity and security objectives.

Additionally, collaboration between development, operations, and security teams has proven to be a key factor for effective integration of security into the pipeline. In [70], structured collaborative practices are shown to facilitate shared vulnerability management, enhance cross-functional communication, and strengthen the overall security posture without compromising operational efficiency.

This approach aligns with the philosophy of shift-left security, which emphasizes the need to incorporate security controls from the design, coding, and planning stages. In [68], an architecture is proposed that integrates threat analysis, early dependency validation, and compliance policies from the earliest phases of the DevOps pipeline, significantly reducing the propagation of vulnerabilities to later stages.

Moreover, [65] proposes an integrated approach for the automated management of security findings within CI/CD pipelines, enabling classification, prioritization, and automated response activation for vulnerabilities detected by SAST and DAST tools. This practice reduces remediation latency and improves the traceability of corrective actions without manual intervention.

There is also a growing trend toward the comprehensive protection of the software supply chain through the use of tools for dependency analysis, third-party image and code scanning [20], [28], [35], [44], [46], and the implementation of automated controls aligned with regulatory frameworks such as NIST and ISO/IEC 27001, ensuring compliance without compromising operational speed [39], [47], [48], [51]. These measures represent robust practices, supported by scientific evidence, that enhance security posture while preserving the agility and operational efficiency characteristic of DevOps environments.

This orientation has also extended to Operational Technology (OT) environments, where operational continuity, physical safety, and regulatory compliance are essential requirements. A tailored DevSecOps model has been proposed that enables the integration of automated controls alongside strategic manual validations, compatible with the technical constraints of critical industrial systems [63].

In highly regulated contexts, implementing DevSecOps requires additional corrective measures to ensure regulatory compliance without compromising automation. In [67], best practices are documented for adapting DevSecOps to sectors such as healthcare or finance, proposing hybrid controls that integrate automation with selective manual validations.

Table 7 summarizes the corrective measures for mitigating vulnerabilities in DevOps environments. The table is structured as follows: from left to right, it presents the corrective measure, related vulnerability, description, DevOps lifecycle application, expected impact, and references indicating the scientific articles where they are discussed.

6) EFFECTS AND IMPLICATIONS OF SECURITY MITIGATION STRATEGIES ON DEVOPS PERFORMANCE

The analysis shows that various mitigation strategies applied in DevOps environments have distinct effects on system performance, creating a balance between cybersecurity protection and operational efficiency. The most effective strategies are those that natively integrate security into the software lifecycle, such as automated security controls, Infrastructure as Code (IaC), automated secret management, and the principle of least privilege with segmented RBAC [13], [14], [17], [26], [30], [45], [50]. These practices help reduce manual errors, ensure consistency across distributed environments, and maintain stable deployment times without introducing significant latency or computational overhead [31], [33], [47].

Recent proposals emphasize the importance of adapting security management to agile frameworks, incorporating lightweight yet effective practices that meet security requirements without causing friction in short delivery cycles. These adaptations enable the DevSecOps model to align with iterative methodologies, improving both delivery speed and software quality in dynamic environments [73].

In terms of performance, mechanisms such as monitoring as code, the use of secure containers, and the instrumentation of automated testing in CI/CD pipelines help preserve system availability and quality without penalizing delivery speed [14], [25], [30], [38], [42], [46], [52]. In parallel, more advanced strategies such as the application of artificial intelligence in DevSecOps, continuous risk assessment, and compliance-as-code models demonstrate high adaptability and accuracy in threat detection. However, they require careful resource management to avoid operational overhead in highly dynamic scenarios [34], [39], [47], [48], [51].

In this regard, [72] explore how artificial intelligence can enhance automation within the DevSecOps lifecycle, proposing models for autonomous threat detection, dynamic security policy generation, and contextual adjustment of controls based on pipeline behavior. Their study shows improvements in both response capacity and reduction of operational overhead in dynamic environments.

Finally, hybrid approaches—such as the combination of manual and automated controls, the adoption of immutable virtual machines, and specialized security training for developers—prove complementary in strengthening security posture without degrading performance [19], [24], [27], [36], [48], [51]. Collectively, the findings reinforce the importance of adopting mitigation strategies that are not only technically

TABLE 7. Corrective measures to mitigate vulnerabilities in DevOps environments.

Corrective Measure	Related Vulnerability	DevOps Lifecycle Phase	Expected Contribution	References
Integration of automated security testing	Lack of automated security testing	Development, Integration, Testing	Early vulnerability identification and risk reduction in production.	[13][17][26][30][38][42][46][52]
Secure secret management	Inadequate secret management in CI/CD	Integration, Deployment, Operations	Unauthorized access prevention and regulatory compliance.	[13][14][20][45][50]
Configuration validation in IaC and containers	Misconfigurations in containers	Build, Integration, Deployment	Reduction of human errors and secure implementation consistency.	[14][17][26][28][31][47]
DevSecOps: security from the start	Late security integration	Planning, Development, Integration	Proactive threat mitigation and enhanced security posture.	[12][21][24][27][68]
Continuous analysis and monitoring	Lack of real-time cybersecurity monitoring	Operations, Maintenance	Immediate incident response and persistent threat detection.	[25][34][41][59]
Supply chain vulnerability evaluation	Use of vulnerable dependencies	Build, Deployment	Prevention of exploitation through compromised components.	[20][28][35][44][60]
Access control policies and least privilege	Poor management of privileged access	Integration, Deployment, Production	Reduced attack surface and limited internal threat damage.	[13][21][45][50][61]
Automated security finding management	Latency in vulnerability response	Integration, Validation, Deployment	Reduced exposure time and response without human intervention.	[65]

effective but also aligned with the automated, distributed, and continuous nature of modern DevOps environments.

Table 8 presents the effects and implications of security mitigation strategies on the performance of DevOps environments. The table is structured as follows: from left to right, it includes the mitigation strategy, its impact on system performance, its effectiveness level in applied scenarios, and the references that support the scientific discussion.

Highly effective strategies significantly reduce risks, integrate into CI/CD pipelines, and align with regulatory frameworks [74], [75], [76]. Medium-effectiveness measures provide partial risk coverage and require complementary controls, while low-effectiveness strategies are reactive, difficult to scale, and disconnected from DevSecOps practices, limiting their real-world impact [74], [75], [76].

7) CORRELATION BETWEEN THREATS, VULNERABILITIES, ASSOCIATED RISK, AND THEIR IMPLICATIONS ON THE SECURITY TRIAD

To strengthen the security analysis in DevOps environments, the MAGERIT methodology (Methodology for Information Systems Risk Analysis and Management) [77] was applied to rigorously assess the risks associated with the mitigation strategies identified in the literature. This methodology enabled the establishment of a correlation between detected threats and vulnerabilities and their impact on the fundamental principles of information security: confidentiality, integrity, availability, authentication, and traceability.

The evaluation included risk estimation by combining the likelihood of threat occurrence with the potential impact on information assets, facilitating an objective prioritization of the applied controls. Additionally, the ability of each strategy to mitigate critical incidents affecting operational continuity, credential protection, regulatory compliance, and deployment chain security was analyzed.

The application of MAGERIT provided a structured and reproducible framework that complements the technical-operational focus of mitigation strategies, allowing their effectiveness to be assessed not only in terms of performance but also from a comprehensive perspective based on cyber risk management.

Table 9 presents the correlation between threats, vulnerabilities, and associated risk, as well as their effects and implications on confidentiality, integrity, availability, authentication, and traceability.

The classification of the impact of threats in DevOps environments on the components of the security triad (Confidentiality, Integrity, Availability), as well as on Authentication and Traceability, was based on empirical evidence extracted from the corpus of analyzed scientific articles and supported by risk assessment methodologies such as OWASP Risk Rating, NIST SP 800-30, and MAGERIT. Threats related to poorly implemented secret management in CI/CD pipelines, for instance, were rated as having a high impact on confidentiality and authentication, as they allow unauthorized access to credentials, tokens, and protected artifacts [13], [14], [50]. Similarly, misconfigurations in IaC and containers were rated as high impact on availability and integrity, since errors in automated infrastructure can disrupt services, generate unsafe execution conditions, and facilitate remote attacks [14], [17], [26].

Meanwhile, threats such as the use of unauthorized tools (Shadow IT) or the lack of traceability in code changes were associated with a high impact on traceability,

TABLE 8. Impact of security mitigation strategies on DevOps performance.

Mitigation Strategy	System Performance Impact	Effectiveness Level	References
Security automation and early integration (DevSecOps)	Reduces post-deployment errors while maintaining lifecycle agility	High: improves security without compromising speed	[14][8][17][26][30]
Continuous monitoring and monitoring-as-code	No significant latency, enables rapid incident response	High: proactive detection without excessive overhead	[8][20][24][30]
Policy-as-Code	Eliminates administrative bottlenecks, maintains regulatory consistency	High: reduces manual errors and accelerates compliance	[12][15]
Infrastructure as Code (IaC)	Enhances environment stability and consistency without resource burden	High: prevents misconfigurations and enables scalability	[7][15]
Automated testing and security gates in CI/CD	Integrates smoothly, reinforces software quality	High: mitigates risk before production	[17][18]
Automated secret and credential management	Optimizes authentication without human error	High: improves availability and access protection	[10]
Secure containers (e.g., SecDocker)	Maintains deployment speed, improves isolation	High: reduces attack surface in orchestrated environments	[8]
AI applied to DevSecOps	Moderate resource load, enhances threat detection precision	High: enables real-time contextual response	[16]
Least privilege principle and segmented RBAC	Does not interfere with authentication, improves access control	High: blocks unauthorized escalation without friction	[13][6]
Native Kubernetes security	Proper configuration ensures stable and secure performance	High: improves orchestration and resource isolation	[4]
Immutable virtual machines	Preserves integrity post-deployment; no post-maintenance needed	High: avoids post-production vulnerabilities	[28][29]
Slide-Block framework with pattern-based authentication	Reduces auth time and improves availability	High: improves detection and resilience with low impact	[21][11]
Security as Code	Avoids manual configuration errors; integrates seamlessly	High: consistent policies without manual intervention	[31][5]
Monitoring-as-Code	Automates monitoring without performance penalty	High: improves operational security with low overhead	[14][32]
Continuous risk assessment (FAIR)	Optimizes resource allocation, avoids degradation	High: enables proactive prioritization	[16][23]
Compliance-as-Code	Simplifies audits and reduces manual load	High: maintains regulatory conformity without friction	[17][5][16]
Hybrid controls (manual and automated)	Balances flexibility and automation; delivery speed unaffected	Medium-High: context-dependent effectiveness	[18][10]
VeriDevOps (automated monitors and formal tests)	Reduces risks with minimal operational impact	High: integrates continuous detection and validation	[20][30]
Security training for developers	Improves code quality and reduces design flaws	High: strengthens resilience with no computational cost	[17][8]
Agentless credential management solutions	Improves scalability with minimal overhead	High: enables secure centralized administration	[13][7]

as they hinder auditability, digital forensics, and version control processes [41], [48], [54]. Regarding the probability of occurrence, threats widely documented in real-world environments and frequently reported in pipelines (e.g., privilege escalation through embedded credentials or malware persistence in pipelines without continuous auditing) were rated as high probability, considering their prevalence and ease of exploitation [28], [33], [45]. In contrast, threats with contextual dependencies—such as malicious AI techniques or failures due to lack of security integration in less common tools—were assessed as having medium or low probability.

The assessments presented throughout all tables in this study—including those related to threats, attack vectors, vulnerabilities, mitigation strategies, their effects on performance, and the correlation between threats, vulnerabilities, and associated risks—were established through a rigorous analysis process grounded in technical criteria and expert professional experience in cybersecurity and DevOps environments. These evaluations integrate empirical evidence from the validated document corpus and expert judgment to determine influence, frequency, criticality, operational consequences, and risk relationships. The categorization into levels such as high, medium, and medium-high reflects a well-supported consensus that ensures both the practical and scientific relevance of the results.

This robust methodological approach guarantees that the conclusions and recommendations are backed by a scientifically sound framework aligned with international standards and industry best practices, while also considering the balance between security, risk, and performance in dynamic DevOps environments.

IV. DISCUSSION

A. THREATS IN DEVOPS ENVIRONMENTS

Figure 3 summarizes the most critical threats identified in the analysis of 62 scientific articles on cybersecurity in DevOps environments. These threats span technical, procedural, and organizational dimensions, reflecting the ongoing tension between deployment speed and continuous security. Although there is broad recognition of common

TABLE 9. Correlation between Threats, Vulnerabilities, and Associated risk on the security triad.

Threat	Vulnerability	C	I	A	Auth	Trace	Risk (Probability and Impact)
CI/CD pipeline compromise	Lack of controls in early stages	High	High	Medium	Medium	High	High probability / High impact
Config errors in virtualized envs	Inconsistent manual configurations	Medium	High	High	Medium	Medium	Medium probability / High impact
Secret leakage	Embedded credentials in code/repos	High	High	Medium	High	Medium	High probability / High impact
Code/script injection	Lack of continuous security testing	Medium	High	High	Medium	High	High probability / Medium impact
Non-compliance	Manual control configurations	High	High	Medium	Medium	High	Medium probability / High impact
Supply chain attacks	Unverified container images	High	High	Medium	Medium	Medium	High probability / High impact
Undetected threats	Lack of proactive detection	High	High	High	High	High	High probability / High impact
Audit/compliance failure	Controls outside standard	High	High	Medium	Medium	High	Medium probability / High impact
Persistent vulns in production	Manual updates or missing restart	Medium	High	Medium	Medium	High	Medium probability / Medium impact
Pipeline changes not validated	No auto-validation before deployment	Medium	High	High	Medium	High	Medium probability / High impact
Service segmentation failures	Insecure cluster configuration	High	High	High	High	Medium	High probability / High impact
Critical threat deprioritization	Lack of exposure awareness	Medium	High	High	Medium	High	Medium probability / High impact
Human errors introducing flaws	Lack of security knowledge	Medium	High	Medium	Medium	Medium	High probability / Medium impact
No continuous monitoring	Lack of event visibility	Medium	Medium	High	Medium	High	Medium probability / High impact
Weak authentication mechanisms	Predictable login methods	High	Medium	Medium	High	High	High probability / High impact
Unauthorized access	No centralized access control	High	Medium	High	High	Medium	High probability / High impact
Tool-process misalignment	Unaligned technical/operational security	Medium	High	Medium	Medium	High	Medium probability / Medium impact
Policy inconsistency	Controls not in source code	High	High	Medium	Medium	High	Medium probability / High impact
Use of unauthorized tools	Lack of stack control	Medium	Medium	High	Medium	High	Medium probability / Medium impact
Vulnerable third-party dependencies	Libraries w/o security validation	High	High	Medium	Medium	Medium	High probability / High impact

threats, significant limitations remain in their comprehensive treatment and empirical validation.

One of the most recurrent findings is the exposure of misconfigurations in IaC and containers. Rangnau et al. [14] and Adhikari and Pillai [47] highlight that the lack of structured validation in automated environments results in poorly secured services that are accessible from external networks. Narang and Mittal [44] complement this perspective by demonstrating that default configurations can be exploited without prior authentication. However, Pan et al. [20] focus their analysis on deployment efficiency without addressing the impact of IaC on the attack surface.

Regarding poor management of privileged access, Cifuentes et al. [13] and Mohan and Othmane [21] report that the lack of token rotation, embedded credentials, and absence of RBAC create internal escalation pathways. Akbar and Alsanad [33] reinforce this by showing how weak configurations in integration tools allow uncontrolled lateral access. Nonetheless, studies such as Neharika and Lennon [28] recognize pipeline criticality but omit identity control as a structural threat.

Threats arising from lack of authentication between microservices and inadequate network policies are discussed in [6], [23], [30], and [54], where the absence of mTLS and internal segmentation is shown to facilitate lateral attacker movement. Hrusto et al. [41] propose architectures with cross-authentication as a mitigation measure, while other studies such as Zeng et al. [12] address microservices without incorporating specific authentication controls, limiting the applicability of their proposals.

In terms of the human component, Rangnau et al. [14] and Adhikari and Pillai [47] mention the prevalence of Shadow IT and unsafe manual tasks as frequent threats, particularly in organizations with low DevSecOps maturity. However, many studies such as Mohan and Othmane [21] and Casola et al. [16] focus exclusively on technical aspects

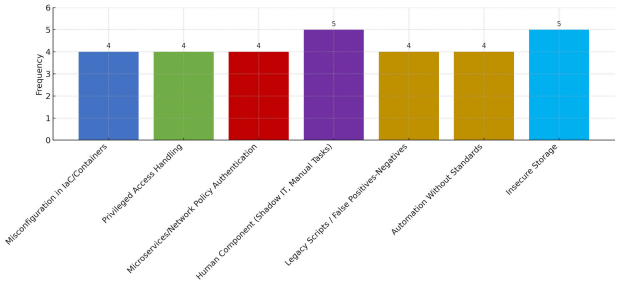

FIGURE 3. Frequency of Identified Threats in DevOps Environments.

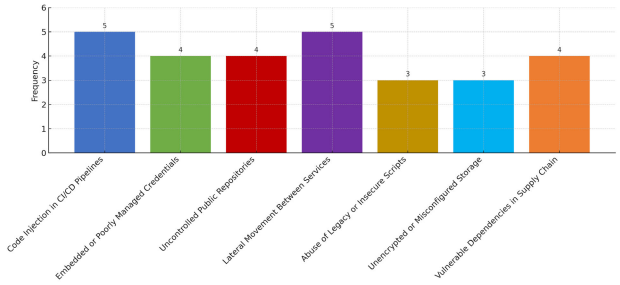

FIGURE 4. Frequency of Identified Attack Vectors in DevOps Environments.

without addressing how organizational culture influences the emergence or persistence of these threats.

There is also weak coverage of threats such as reliance on legacy scripts [14], [26], [28] or false positives/negatives in automated scanners [11], [38], [46], [52]. Rajakumar and Thason [52] propose progressive calibration of SAST and DAST tools, while Sanmorino [35] document how unreviewed legacy scripts introduce silent failures. However, these proposals lack comparative evaluations or precision and coverage metrics.

Finally, topics such as automation misaligned with compliance standards [39], [47], [48] and persistence of data in unencrypted storage [44], [46], [49] are addressed from a normative perspective. Khan et al. [60] propose a verified automation model using compliance policies, but other studies like Casola et al. [51] only mention these threats without offering a formal mitigation framework. This reveals a disconnect between problem recognition and structured resolution.

B. ATTACK VECTORS IN DEVOPS ENVIRONMENTS

Figure 4 shows the most frequently identified attack vectors from the analysis of the scientific literature. These vectors represent specific technical pathways through which attackers exploit the threats described in DevOps environments. While there is general consensus regarding the main vectors, the depth of treatment and validation of the proposed approaches varies significantly across studies.

One of the most frequently reported vectors is the injection of malicious code into CI/CD pipelines, as discussed in [13], [26], [28], [33], and [40]. Cifuentes et al. [13] explain how the lack of script validation allows arbitrary code execution in automated environments. Nadeem and Shameem [26] and Neharika and Lennon [28] propose mitigating this risk through digital signatures and pipeline segmentation, while Zhang and Zhang [40] present quantitative metrics on CI/CD security. However, most studies do not empirically validate the effectiveness of these proposals.

Another critical vector is the use of embedded credentials or poorly managed secrets. Cifuentes et al. [13] and Rangnau et al. [14] show how such practices allow persistent access to critical resources within the pipeline. Khan et al. [60] suggest using external vaults such as HashiCorp Vault, whereas other studies [30] still omit secret management as an essential component of a secure DevOps model.

Direct access to public repositories without control mechanisms is highlighted in [13], [14], [20], and [30]. These vectors facilitate the exposure of sensitive source code or hardcoded credentials. Rangnau et al. [14] propose restricting access through version control and visibility analysis, although no metrics are reported on their effectiveness in real-world deployments.

Lateral movement across unsegmented services, caused by inadequate network policies, is described in [3], [6], [18], [21], and [54]. Hrusto et al. [41] propose the use of mutual authentication and domain-based segmentation to mitigate this attack path. In contrast, studies such as [17] focus on microservices without incorporating effective isolation or traffic control mechanisms.

With regard to legacy script abuse, Sanmorino [35] and Nadeem and Shameem [26] warn that insecure commands, broad permissions, or unaudited structures can lead to persistent and uncontrolled executions. However, other works such as [45] acknowledge their existence without linking them directly to explicit attack vectors.

The exposure of unencrypted storage or misconfigured buckets is addressed by Lee et al. [46] and Narang and Mittal [44], who document cases of sensitive data leakage without requiring elevated privileges. Nevertheless, several studies omit storage as an active attack surface and do not propose specific controls for its protection or continuous monitoring.

Finally, vectors associated with the software supply chain, such as the inclusion of vulnerable dependencies, are identified in [14], [28], [44], and [57]. While some articles recommend the use of Software Composition Analysis (SCA) tools, only Khan et al. [60] provide an architecture that integrates automated component scanning and cryptographic validation. This proposal, however, has not been replicated or compared with alternative approaches in the remaining studies.

C. METHODOLOGICAL APPROACHES FOR THREAT MITIGATION IN DEVOPS ENVIRONMENTS

The analysis of the scientific literature reveals growing concern over the integration of mitigation mechanisms aligned with the threats and attack vectors present in DevOps

environments. However, although many publications identify effective practices, few present systematic empirical validations or comparative assessments between strategies. Figure 5 summarizes the most frequently documented approaches.

One dominant methodological line is the automation of security testing and static code analysis in CI/CD pipelines. Lee et al. [46] and Enoiu et al. [30] highlight that integrating SAST scanners at each commit reduces the exposure time of vulnerabilities and enables proactive code verification before deployment. Nevertheless, studies such as Sanmorino [35] implement this practice without measuring precision metrics, such as false positive rates or coverage levels.

With regard to Infrastructure as Code (IaC) validation, Dasanayake et al. [19] and Adhikari and Pillai [47] propose tools such as OPA and Terraform Validator to ensure secure configurations before deployment. Despite their popularity, Pan et al. [20] do not address these techniques, focusing instead on IaC from an operational efficiency perspective. This reflects a bias in the literature favoring functionality over structural security.

Automated secret management is another key mitigation area. Cifuentes et al. [13] and Mohan and Ben Othmane [21] promote the adoption of vaults such as HashiCorp Vault and periodic token rotation. Khan et al. [60] advocate for expiration policies on embedded credentials, but several studies, such as [28] and [30], mention best practices without detailing their practical implementation or validating their impact on incident reduction.

Regarding mutual authentication and network segmentation, Narang and Mittal [44] and Hrusto et al. [41] propose controls such as mTLS between microservices and extended RBAC. However, only Khan et al. [60] describe a comprehensive model combining segmentation, observability, and distributed authentication, validated through experimental data.

Continuous monitoring and real-time response approaches are less common but highly relevant. Sadovykh et al. [17] and Rajakumar and Thason [52] address behavior-based rules and SIEM tools as early detection methods. Still, many articles treat observability superficially, without exploring how operational data can effectively inform security processes.

Finally, supply chain protection strategies—such as the use of SCA tools, artifact signing, and version control—are recommended by Khan et al. [60] and Rangnau et al. [14]. Akbar and Alsanad [33] mention the need for dependency verification but do not propose concrete tools or evaluate practical scenarios.

D. VULNERABILITIES IN DEVOPS ENVIRONMENTS

The analysis of the scientific literature enabled the identification of a robust set of recurring vulnerabilities in DevOps environments. These vulnerabilities encompass both technical weaknesses—such as misconfigurations and reliance on insecure components—and failures in development processes, automation, and organizational culture. Although there is broad consensus on their criticality, there is a notable lack of empirical validation to measure the real impact of each type of vulnerability in operational contexts. Figure 6 summarizes the details of the vulnerabilities.

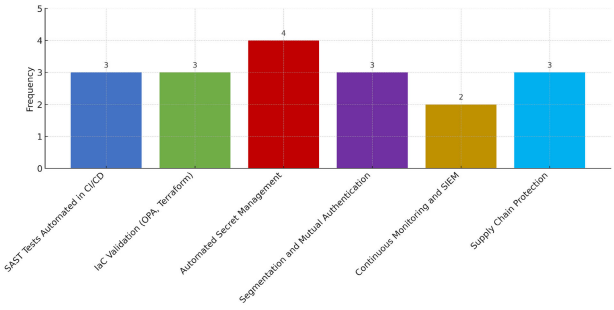

FIGURE 5. Mitigation Strategies in DevOps Environments.

One of the most widely documented vulnerabilities is the absence of automated security testing. Gartner [5] and Enoiu et al. [30] show how the lack of SAST and DAST controls in CI/CD pipelines allows critical errors to reach production. Sanmorino [35] and Rangnau et al. [14] also agree that this deficiency is widespread across many DevOps projects, although their studies do not provide tool comparisons or efficacy metrics.

Another critical vulnerability is the exposure of confidential secrets, especially in public repositories or environments lacking secure management. Yilmaz and Harding [9] demonstrate how embedded tokens without rotation are common vectors for unauthorized access. Additionally, inadequate secret management in CI/CD [7], [9], [13], [14], [20], [21] compromises sensitive environments when specific tools such as HashiCorp Vault or AWS Secrets Manager are not integrated.

Insecure configurations in containers and DevOps tools are addressed in studies such as [13], [17], [26], and [29]. Lee et al. [46] report that containers with elevated privileges, no resource limits, or exposure to public networks pose significant threats. However, few works offer concrete guidelines on image scanning or automatic enforcement of security policies.

Vulnerabilities related to poor development practices are also highlighted, such as the lack of secure coding, the use of legacy scripts, or delayed security integration. Dasanayake et al. [19] and Rajakumar and Thason [52] note that these operational weaknesses stem from a misalignment between development and security teams, which hinders the adoption of mature DevSecOps models.

Regarding external dependencies, Khan et al. [60] and Akbar and Alsanad [33] document that the use of unverified or outdated libraries creates entry points for attackers. Despite the latent threat, many studies do not address the use of tools such as Software Composition Analysis (SCA), nor do they offer coverage metrics on their outcomes.

Other underexplored vulnerabilities include security-agnostic automation [5], [12], [14] and the lack of real-time cybersecurity monitoring [34], [36], [41], [43]. Although these issues are identified, the literature does not establish

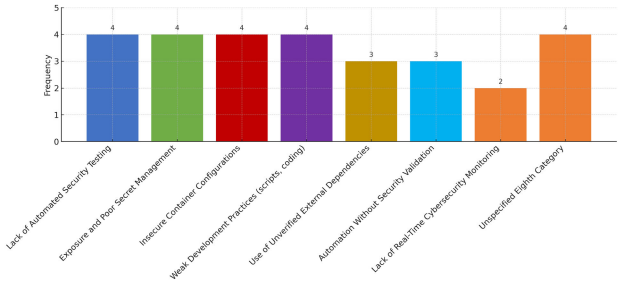

FIGURE 6. Frequency of Identified Vulnerabilities in DevOps Environments.

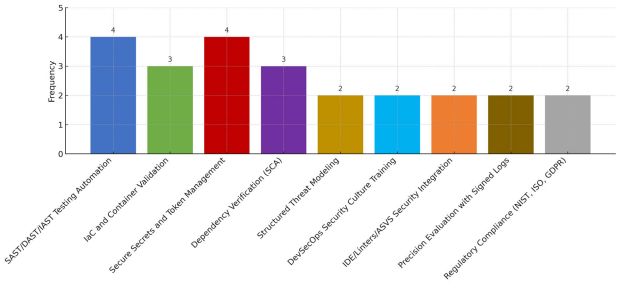

FIGURE 7. Corrective Measures to Mitigate Vulnerabilities in DevOps Environments.

correlations between their presence and the incidents that could have been prevented through early detection.

E. CORRECTIVE MEASURES TO MITIGATE VULNERABILITIES

Figure 7 summarizes the fundamental corrective measures adopted to mitigate vulnerabilities in DevOps environments, which include both technical mechanisms and organizational practices. One of the most widely documented strategies is the automation of security testing through tools such as SAST, DAST, and IAST integrated into CI/CD pipelines. Akbar and Alsanad [33] and Satapathy et al. [15] demonstrate that these techniques help identify critical vulnerabilities early in the software development lifecycle.

However, studies such as Pan et al. [20] confirm that in fast-paced deployment contexts, these tests are often reactive, reducing their effectiveness. Complementarily, configuration validation in IaC and containers has been promoted by Nadeem and Shameem [26] and Aparo et al. [29] as an effective means of avoiding default configurations and minimizing manual errors, although other studies such as Sadovykh et al. [17] reveal a lack of specific validation controls.

Secret and token management is another recurring corrective measure. Yilmaz and Harding [9] propose the use of vaults and granular access control as key mechanisms to prevent accidental exposures, while Casola et al. [51] acknowledge their importance but report adoption barriers in less mature environments. Narang and Mittal [44] and Neharika and Lennon [28] discuss the verification of third-party dependencies through SCA tools such as Snyk or OWASP Dependency Check, but only the former presents empirical evidence of application.

This difference highlights a gap between risk awareness and effective mitigation. Some approaches, such as structured threat modeling (using STRIDE or MITRE), are addressed by Rajakumar and Thason [52] and Lombardi and Fanton [25], who show that early integration improves the ability to anticipate attack vectors. However, this remains absent in articles focused solely on integration or automation.

Similarly, continuous training and the promotion of a DevSecOps culture—emphasized by Mohan and Ben Othmane [21]—are mentioned only superficially in studies such as Enoiu et al. [30], limiting the adoption of best practices such as the principle of least privilege or collaborative code reviews.

Moreover, measures such as integrating security into IDEs and pipeline validators, using automated linters and standards like OWASP ASVS, and regularly updating base images are scarcely discussed in the literature despite their potential to reduce logic flaws or inherited vulnerabilities. Only a few studies, such as Cifuentes et al. [13] and Sadovykh et al. [17], address these practices with verifiable data.

Additionally, techniques such as behavior analysis in pipelines, precision assessment of SAST tools, or exclusive use of signed private registries are only marginally represented, despite being critical to mitigating persistent threats and ensuring artifact integrity. Finally, only a small subset of articles, such as those by Casola et al. [51] and Lombardi and Fanton [25], succeed in integrating these approaches into formal compliance frameworks such as NIST SP 800-53, ISO/IEC 27001, or GDPR by explicitly mapping automated controls to regulatory requirements.

This lack of systemic alignment limits the impact of many of these measures in real-world environments, reaffirming the need to adopt a holistic, automated, and compliance-oriented approach to effectively mitigate vulnerabilities in DevSecOps environments.

F. EFFECTS AND IMPLICATIONS OF SECURITY MITIGATION STRATEGIES ON THE PERFORMANCE OF DEVOPS ENVIRONMENTS

In general, the reviewed studies agree that early integration of security through DevSecOps practices enhances the defensive posture without compromising delivery speed. For instance, Garcia and Bartel [66] and Rangnau et al. [14] report a significant reduction in post-deployment errors without affecting the agile development cycle. However, Sadovykh et al. [17] warn that, although this integration proves highly effective, it also requires a learning curve for technical teams unfamiliar with automation and monitoring tools.

Strategies such as continuous supervision and monitoring-as-code are highlighted by Pan et al. [20] and Nath et al. [24] as low-impact mechanisms that allow real-time incident detection. Nevertheless, their effectiveness largely depends on proper alert calibration, correlation rule tuning, and feedback mechanisms.

Regarding Infrastructure as Code (IaC) and Security as Code, studies by Satapathy et al. [15] and Casola et al. [39] demonstrate that their adoption does not introduce significant overhead and instead promotes operational stability, configuration consistency, and the implementation of replicable and auditable security policies.

More advanced solutions such as VeriDevOps, presented by Sadovykh et al. [17] and Enoiu et al. [30], combine formal verification with automated monitors to enhance security validation during runtime. Although these approaches increase validation coverage, they also require technical investment and adaptation of existing pipelines, which can affect initial adoption in non-specialized teams.

Strategies such as the use of immutable virtual machines [28], secure containers [8], and the principle of least privilege with RBAC [6], [13] are recognized for enhancing structural security without introducing latency or degrading performance in distributed systems.

In contrast, while some publications address native Kubernetes security superficially, studies like Casola et al. [51] recommend incorporating advanced capabilities for isolation, configuration validation, and policy-based access control. Although they acknowledge that full implementation remains limited, these effects and implications are illustrated in Figure 8.

Compliance-as-code has also gained attention for its potential to automate regulatory controls without significant computational impact, as evidenced in the works of Khan et al. [16] and Casola et al. [51], who propose direct alignment with frameworks such as NIST SP 800-53 and ISO/IEC 27001.

Finally, traditionally underestimated practices such as continuous security training show positive effects on code quality and design flaw prevention. Rangnau et al. [14] and Sadovykh et al. [17] demonstrate that an active DevSecOps culture can strengthen the security posture without compromising operational performance.

G. CORRELATION BETWEEN THREATS, VULNERABILITIES, ASSOCIATED RISK, AND THEIR IMPLICATIONS ON THE SECURITY TRIAD

Table 9 summarizes the correlation between frequent threats in DevOps environments, the underlying vulnerabilities that enable them, and the level of impact on the fundamental pillars of information security: confidentiality, integrity, availability, authentication, and traceability. Additionally, an estimation of the associated risk is included, calculated based on the likelihood of occurrence and the projected impact in real operational scenarios.

Threats that exhibit a high-risk level—characterized by both high likelihood and high impact—tend to simultaneously compromise multiple dimensions of security. The most critical include CI/CD pipeline compromise, exposure of confidential secrets, software supply chain attacks, and the absence of proactive detection mechanisms. These conditions not only degrade the integrity of the environment but also undermine traceability and confidentiality, impeding forensic reconstruction and limiting post-incident auditing capabilities.

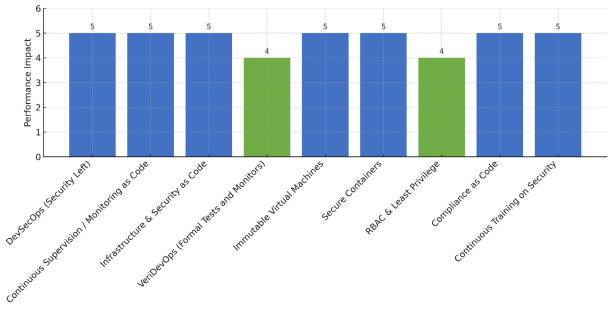

FIGURE 8. Effects and Implications of Security Mitigation Strategies in DevOps Environments.

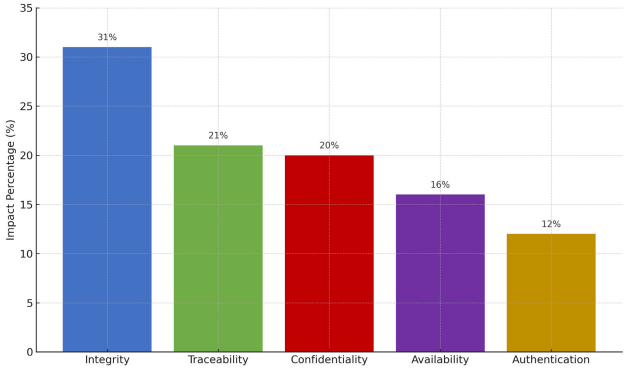

FIGURE 9. Correlation Between Threats, Vulnerabilities, and Associated Risk.

Figure 9 visualizes the distribution of severe impacts across each security principle, based on the threat analysis. The data show that integrity is the most affected component (31%), followed by traceability (21%) and confidentiality (20%). This concentration of vulnerabilities in structural dimensions suggests that attacks in DevOps environments aim not only to disrupt availability, but also to alter system behavior and evade monitoring mechanisms. This emphasizes the urgency of implementing controls such as automated IaC validation, continuous event auditing, and role-based access segmentation.

In terms of composite risk, most threats classified as critical affect multiple security pillars simultaneously, demanding holistic and well-integrated mitigation approaches. Consequently, cybersecurity management in DevOps environments must go beyond isolated controls and shift toward a resilient architecture supported by integrated policies, continuous monitoring, strong authentication, and rigorous management of external dependencies and trusted execution environments.

V. CONCLUSION AND FUTURE WORK

This systematic literature review has provided a comprehensive view of the challenges, limitations, and solution patterns related to cybersecurity in DevOps environments. Based on a rigorous analysis of 62 scientific articles published between

2016 and 2025, the study successfully mapped the critical components of the DevSecOps ecosystem, identifying recurrent threats, active attack vectors, structural vulnerabilities, documented mitigation strategies, and their technical impact on performance and operational resilience.

The findings show that the most critical threats are linked to unsupervised automation, secret exposure in CI/CD pipelines, lack of mutual authentication among distributed services, software supply chain attacks, and the use of unauthorized tools (Shadow IT). These threats simultaneously compromise key pillars such as integrity, traceability, confidentiality, and availability. From a technical perspective, the most frequent attack vectors include code injection in pipelines, uncontrolled access to public repositories, remote execution due to default configurations, and lateral movement in unsegmented architectures.

The vulnerability analysis enabled the identification of 27 technical and organizational weaknesses, notably the lack of automated security testing, poor secret management, reliance on insecure legacy scripts, and the use of unverified dependencies. These vulnerabilities affect all phases of the DevOps cycle—from design to operations. In response, over 30 mitigation strategies were documented, including technical approaches such as SAST/DAST/IAST scanning, validated IaC, secret management through vaults, RBAC-based access control, and organizational frameworks such as DevSecOps, VeriDevOps, and Compliance-as-Code.

A key finding of this study is that automated mitigation strategies, when properly aligned with the DevOps cycle, do not degrade system performance; on the contrary, they strengthen stability, reduce the attack surface, and improve incident response capabilities. However, a critical limitation was identified in the literature: most reviewed studies lack rigorous empirical validation. In many cases, the proposed measures have not undergone controlled testing, nor are there comparative effectiveness metrics in productive contexts. This gap between declarative and effective security represents a challenge that must be urgently addressed from a research perspective.

Based on these findings, and recognizing the growing adoption of edge, distributed, and sensor-based environments, the Internet of Things (IoT) emerges as a natural extension for this research. The complexity of IoT environments—marked by device heterogeneity, intermittent connectivity, limited resources, and the need for secure remote updates—introduces new pressures on the traditional DevSecOps model. In this regard, cybersecurity in DevOps environments must evolve to address architectures where the perimeter is not clearly defined and where security must be contextual, continuous, and intervention-free.

As a research projection, it is recommended to explore the feasibility of integrating SAST/DAST scanning and firmware validation into pipelines adapted to IoT devices, the use of blockchain for distributed traceability, secure lifecycle management of IoT through Policy-as-Code, and telemetry-based feedback. Likewise, fertile ground exists for evaluating the energy efficiency of security measures in low-power IoT devices, as well as analyzing human factors and their impact on cybersecurity incidents from a DevOps perspective.

Finally, a complementary research direction involves the design of comparative empirical evaluation frameworks to validate DevSecOps strategies in hybrid contexts (cloud-edge-IoT), incorporating technical, organizational, and regulatory dimensions. Such efforts would not only consolidate the theoretical contribution of this review but also enable the development of new resilient models applicable to evolving automated and distributed critical infrastructures. Likewise, promising opportunities are identified for applying DevSecOps principles to domains of critical infrastructure, such as Operational Technology (OT) and industrial automation environments. These scenarios share key characteristics with DevOps—such as the need for automation, orchestration, and operational continuity—but impose additional constraints on availability and physical security.

REFERENCES

[1] D. Farley and J. Humble, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Reading, MA, USA: Addison-Wesley, 2011.

[2] I. M. Weber, L. Bass, and L. Zhu, *DevOps: A Software Architect's Perspective*. Reading, MA, USA: Addison-Wesley, 2015.

[3] G. Kim, J. Humble, and P. D. Y. J. Willis, *The DevOps Handbook: How To Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland, OR, USA: IT Revolution Press, 2016.

[4] A. A. U. Rahman and L. Williams, "Software security in DevOps: Synthesizing Practitioners' perceptions and practices," in *Proc. IEEE/ACM Int. Workshop Continuous Softw. Evol. Del. (CSED)*, May 2016, pp. 70–76, doi: 10.1145/2896941.2896946.

[5] *Innovation Insight for DevSecOps*, Gartner Inc., Stamford, CT, USA, 2017.

[6] M. A. Babar, "Security in DevOps: State-of-the-art and future directions," *IEEE Softw.*, vol. 35, no. 5, pp. 92–96, Sep. 2018, doi: 10.1016/j.infsof.2021.106700.

[7] Cybersecurity and Infrastructure Security Agency (CISA). (2021). *Remediating the SolarWinds Supply Chain Attack*. [Online]. Available: https://www.cisa.gov/news-events/alerts/2021/04/22/remediating-solarwinds-supply-chain-attack

[8] The Free and Open Source Software DevSecOps Team, "Supply chain attacks in CI/CD systems: A systematic analysis," *IEEE Secur. Privacy*, vol. 20, no. 1, pp. 38–46, Jan./Feb. 2022, doi: 10.1109/MSEC.2021.3134107.

[9] B. Kitchenham and Y. S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Dept. Comput. Sci., Keele Univ., Univ. Durham, Staffordshire, U.K., Tech. Rep. EBSE-2007-01, 2007.

[10] J. Vehent, *Securing DevOps: Security in the Cloud*. Sebastopol, CA, USA: O'Reilly Media, 2018. [Online]. Available: https://www.oreilly.com/library/view/securing-devops/9781617294136/

[11] S. Jalali and C. Wohlin, "Systematic literature studies: Database searches vs. Backward snowballing," in *Proc. ACM-IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Sep. 2012, pp. 29–38, doi: 10.1145/2372251.2372257.

[12] Q. Zeng, M. Kavousi, Y. Luo, L. Jin, and Y. Chen, "Full-stack vulnerability analysis of the cloud-native platform," *Comput. Secur.*, vol. 129, Jun. 2023, Art. no. 103173, doi: 10.1016/j.cose.2023.103173.

[13] C. Cifuentes, F. Gauthier, B. Hassanshahi, P. Krishnan, and D. McCall, "The role of program analysis in security vulnerability detection: Then and now," *Comput. Secur.*, vol. 135, Dec. 2023, Art. no. 103463, doi: 10.1016/j.cose.2023.103463.

[14] T. Rangnau, R. V. Buijtenen, F. Fransen, and F. Turkmen, "Continuous security testing: A case study on integrating dynamic security testing tools in CI/CD pipelines," in *Proc. IEEE 24th Int. Enterprise Distrib. Object Comput. Conf. (EDOC)*, Oct. 2020, pp. 145–154, doi: 10.1109/EDOC49727.2020.00026.

[15] U. Satapathy, R. Thakur, S. Chattopadhyay, and S. Chakraborty, "DisProTrack: Distributed provenance tracking over serverless applications," in *Proc. IEEE Conf. Comput. Commun.*, May 2023, pp. 1–10, doi: 10.1109/INFOCOM53939.2023.10228884.

[16] V. Casola, A. De Benedictis, M. Rak, and U. Villano, "A novel security-by-design methodology: Modeling and assessing security by SLAs with a quantitative approach," *J. Syst. Softw.*, vol. 163, May 2020, Art. no. 110537, doi: 10.1016/j.jss.2020.110537.

[17] A. Sadovykh, G. Widforss, D. Truscan, E. P. Enoiu, W. Mallouli, R. Iglesias, A. Bagnto, and O. Hendel, "VeriDevOps: Automated protection and prevention to meet security requirements in DevOps," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Grenoble, France, Feb. 2021, pp. 1330–1333, doi: 10.23919/DATE51398.2021.9474185.

[18] M. A. Aljohani and S. S. Alqahtani, "A unified framework for automating software security analysis in DevSecOps," in *Proc. Int. Conf. Smart Comput. Appl. (ICSCA)*, Kuala Lumpur, Malaysia, Feb. 2023, pp. 1–6, doi: 10.1109/ICSCA57840.2023.10087568.

[19] S. D. L. V. Dasanayake, J. Senanayake, and W. M. J. I. Wijayanayake, "DevSecOps implementation for continuous security in financial trading software application development," in *Proc. 5th Int. Conf. Adv. Res. Comput. (ICARC)*, Colombo, Sri Lanka, Feb. 2025, pp. 1–6, doi: 10.1109/icarc64760.2025.10963292.

[20] Z. Pan, W. Shen, X. Wang, Y. Yang, R. Chang, Y. Liu, C. Liu, Y. Liu, and K. Ren, "Ambush from all sides: Understanding security threats in open-source software CI/CD pipelines," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 403–418, Jan. 2024, doi: 10.1109/TDSC.2023.3253572.

[21] V. Mohan and L. B. Othmane, "SecDevOps: Is it a marketing buzzword?—Mapping research on security in DevOps," in *Proc. 11th Int. Conf. Availability, Rel. Secur. (ARES)*, Aug. 2016, pp. 542–547, doi: 10.1109/ARES.2016.92.

[22] A. Valani, "Rethinking secure DevOps threat modeling: The need for a dual velocity approach," in *Proc. IEEE Cybersecurity Develop. (SecDev)*, Cambridge, MA, USA, Sep. 2018, pp. 136–143, doi: 10.1109/SECDEV.2018.00032.

[23] K. Tuma, G. Calikli, and R. Scandariato, "Threat analysis of software systems: A systematic literature review," *J. Syst. Softw.*, vol. 144, pp. 275–294, Oct. 2018, doi: 10.1016/j.jss.2018.06.073.

[24] P. Nath, J. R. Mushahary, U. Roy, M. Brahma, and P. K. Singh, "AI and blockchain-based source code vulnerability detection and prevention system for multiparty software development," *Comput. Electr. Eng.*, vol. 106, Feb. 2023, Art. no. 108607, doi: 10.1016/j.compeleceng.2023.108607.

[25] F. Lombardi and A. Fanton, "From DevOps to DevSecOps is not enough. CyberDevOps: An extreme shifting-left architecture to bring cybersecurity within software security lifecycle pipeline," *Softw. Quality J.*, vol. 31, no. 2, pp. 619–654, Jun. 2023, doi: 10.1007/s11219-023-09619-3.

[26] A. Kumar, M. Nadeem, and M. Shameem, "Metaheuristic-based cost-effective predictive modeling for DevOps project success," *Appl. Soft Comput.*, vol. 163, Sep. 2024, Art. no. 111834, doi: 10.1016/j.asoc.2024.111834.

[27] S. Dupont, P. Massonet, G. Ginis, and C. Ponsard, "Product Incremental Security Risk Assessment Using DevSecOps Practices," in *Proc. European Symp. Res. Comput. Secur.*, vol. 13785. Cham, Switzerland: Springer, 2023, pp. 603–620, doi: 10.1007/978-3-031-25460-4_38.

[28] K. Neharika and R. G. Lennon, "Investigations into Secure IaC Practices," in *Proc. ICICT*, vol. 448. Springer, 2023, pp. 289–303, doi: 10.1007/978-981-19-1610-6_25.

[29] C. Aparo, C. Bernardeschi, G. Lettieri, F. Lucattini, and S. Montanarella, "An analysis system to test security of software on continuous integration-continuous delivery pipeline," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroS&PW)*, Jul. 2023, pp. 58–67, doi: 10.1109/EUROSPW59978.2023.00012.

[30] E. P. Enoiu, D. Truscan, A. Sadovykh, and W. Mallouli, "VeriDevOps software methodology: Security verification and validation for DevOps practices," in *Proc. 18th Int. Conf. Availability, Rel. Secur.*, Benevento, Italy, Aug. 2023, pp. 1–9, doi: 10.1145/3600160.3605054.

[31] D. Fernández González, F. J. Rodríguez Lera, G. Esteban, and C. Fernández Llamas, "SecDocker: Hardening the continuous integration workflow: Wrapping the container layer," *Social Netw. Comput. Sci.*, vol. 3, no. 1, pp. 1–13, Jan. 2022, doi: 10.1007/s42979-021-00939-4.

[32] T. Okubo and H. Kaiya, "Efficient secure DevOps using process mining and attack defense trees," *Proc. Comput. Sci.*, vol. 207, pp. 446–455, Jan. 2022, doi: 10.1016/j.procs.2022.09.079.

[33] M. A. Akbar and A. A. AlSanad, "Empirical investigation of key enablers for secure DevOps practices," *IEEE Access*, vol. 13, pp. 43698–43715, 2025, doi: 10.1109/ACCESS.2025.3549183.

[34] S. Moreschini, S. Pour, I. Lanese, D. Balouek, J. Bogner, X. Li, F. Pecorelli, J. Soldani, E. Truyen, and D. Taibi, "AI techniques in the microservices life-cycle: A systematic mapping study," *Computing*, vol. 107, no. 4, pp. 1–20, Apr. 2025, doi: 10.1007/s00607-025-01432-z.

[35] A. Sanmorino, "The role of data science in enhancing web security," *JEECS (Journal Electr. Eng. Comput. Sciences)*, vol. 9, no. 2, pp. 119–126, Nov. 2024, doi: 10.54732/jeecs.v9i2.4.

[36] M. Matias, E. Ferreira, N. Mateus-Coelho, and L. Ferreira, "Enhancing effectiveness and security in microservices architecture," *Proc. Comput. Sci.*, vol. 239, pp. 2260–2269, Jun. 2024, doi: 10.1016/j.procs.2024.06.417.

[37] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 146, pp. 215–232, Dec. 2018, doi: 10.1016/j.jss.2018.09.082.

[38] R. Grande, A. Vizcaíno, and F. O. García, "Is it worth adopting DevOps practices in global software engineering? Possible challenges and benefits," *Comput. Standards Interfaces*, vol. 87, Jan. 2024, Art. no. 103767, doi: 10.1016/j.csi.2023.103767.

[39] V. Casola, A. De Benedictis, C. Mazzocca, and V. Orbinato, "Secure software development and testing: A model-based methodology," *Comput. Secur.*, vol. 137, Feb. 2024, Art. no. 103639, doi: 10.1016/j.cose.2023.103639.

[40] J. Y. Zhang and Y. Zhang, "Quantitative DevSecOps metrics for cloud-based web microservices," *IEEE Access*, vol. 12, pp. 160317–160342, 2024, doi: 10.1109/ACCESS.2024.3486314.

[41] A. Hrusto, M. Hafner Y, and R. Breu, "Security risks in modern CI/CD chains: A review," *J. Syst. Softw.*, vol. 23, 2023, Art. no. 000737.

[42] V. Veeramachaneni, "A systematic review of DevSecOps: Bridging security and agile development," *Int. J. Inf. Secur.*, vol. 21, pp. 87–104, Jan. 2022, doi: 10.1007/s10207-021-00549-2.

[43] O. H. Plant, M. T. Johnson, and K. L. Smith, "Rethinking IT governance: Designing a framework for mitigating risk in DevOps," *Inf. Syst. Frontiers*, vol. 26, pp. 311–326, Feb. 2023, doi: 10.1007/s10796-022-10245-7.

[44] P. Narang and P. Mittal, "Performance assessment of traditional software development methodologies and DevOps automation culture," *Eng., Technol. Appl. Sci. Res.*, vol. 12, no. 6, pp. 9726–9731, Dec. 2022, doi: 10.48084/etasr.5315.

[45] H. Takeda, M. Sato, and K. Yamamoto, "Mismanagement of privileged access in DevOps environments," in *Proc. IEEE Int. Conf. Softw. Secur. (ICSS)*, Dec. 2021, pp. 110–119, doi: 10.1109/ICSS.2021.00020.

[46] H. Takeda, M. Sato, and K. Yamamoto, "Mismanagement of Privileged Access in DevOps Environments," in *Proc. IEEE Int. Conf. Comput. Secur. (ICCS)*, Dec. 2021, pp. 110–119, doi: 10.1109/ICCS.2021.00020.

[47] T. Adhikari and S. Pillai, "IaC configuration errors and exploitable patterns," in *Proc. IEEE Cloud Secur. Conf.*, 2023.

[48] L. Müller and T. Koch, "Network misconfiguration threats in microservice-based DevOps," in *Proc. Eur. Symp. Secure Softw. Syst.*, 2023.

[49] M. Bedoya, S. Palacios, D. Díaz-López, E. Laverde, and P. Nespoli, "Enhancing DevSecOps practice with large language models and security chaos engineering," *Int. J. Inf. Secur.*, vol. 23, no. 6, pp. 3765–3788, Dec. 2024, doi: 10.1007/s10207-024-00909-w.

[50] A. S. A. Alghawli and T. Radivilova, "Resilient cloud cluster with DevSecOps security model, automates a data analysis, vulnerability search and risk calculation," *Alexandria Eng. J.*, vol. 107, pp. 136–149, Nov. 2024, doi: 10.1016/j.aej.2024.07.036.

[51] M. Safaei Pour, C. Nader, K. Friday, and E. Bou-Harb, "A comprehensive survey of recent internet measurement techniques for cyber security," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103123, doi: 10.1016/j.cose.2023.103123.

[52] J. R. M. thason, "AI-driven DevSecOps: Advancing security and compliance in continuous delivery pipelines," *Int. Res. J. Adv. Eng. Manage. (IRJAEM)*, vol. 3, no. 5, pp. 1666–1673, May 2025, doi: 10.47392/IRJAEM.2025.0268.

[53] K. Albagami, N. Van Huynh, and G. Ye Li, "A universal deep neural network for signal detection in wireless communication systems," in *Proc. IEEE Int. Conf. Mach. Learn. Commun. Netw. (ICMLCN)*, Stockholm, Sweden, May 2024, pp. 1–6, doi: 10.1109/ICMLCN59089.2024.10624797.

[54] Z. Shahbazi and M. Mesbah, "Deep learning techniques for enhancing the efficiency of security patch management in DevSecOps pipelines," in *Proc. Int. Conf. Softw. Eng.*, 2024, pp. 1–12, doi: 10.1007/978-981-96-5693-6_31.

[55] M. I. Ahmed, *Cloud-Native DevOps: Building Scalable and Reliable Applications*. Cham, Switzerland: Springer, 2024.

[56] D. De Oliveira, J. M. L. Dos Santos, and Y. D. N. Da Silva, "Efficient secure DevOps using process mining and attack defense trees," *Proc. Comput. Sci.*, vol. 207, pp. 4076–4085, Jan. 2022, doi: 10.1016/J.PROCS.2022.12.096.

[57] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Inf. Softw. Technol.*, vol. 141, Jan. 2022, Art. no. 106700, doi: 10.1016/j.infsof.2021.106700.

[58] R. Alwahaishi and H. A. Alharthi, "Slide-block: End-to-end amplified security to improve DevOps resilience," *Heliyon*, vol. 10, no. 2, p. e2430, doi: 10.1016/j.heliyon.2024.e2430.

[59] M. A. Akbar, K. Smolander, S. Mahmood, and A. Alsanad, "Toward successful DevSecOps in software development organizations: A decision-making framework," *Inf. Softw. Technol.*, vol. 147, Jul. 2022, Art. no. 106894, doi: 10.1016/j.infsof.2022.106894.

[60] X. Zhao, T. Clear, and R. Lal, "Identifying the primary dimensions of DevSecOps: A multi-vocal literature review," *J. Syst. Softw.*, vol. 214, Aug. 2024, Art. no. 112063, doi: 10.1016/j.jss.2024.112063.

[61] H. Dubois and B. Fröhlich, "Cybersecurity in a DevOps environment," *Commun. Comput. Inf. Sci.*, vol. 1731, pp. 31–44, doi: 10.1007/978-3-031-42212-6_3.

[62] A. Patel, K. Jain, and D. Singh, "DevOps challenges and risk mitigation strategies by DevOps professional teams," in *Proc. Int. Conf. Softw. Bus.*, vol. 502, pp. 331–343, doi: 10.1007/978-3-031-53227-6_26.

[63] R. Deshmukh, "Information-centric adoption and use of standard compliant DevSecOps for operational technology," in *Proc. Lect. Notes Bus. Inf. Process.*, vol. 502, pp. 355–368, doi: 10.1007/978-3-031-53227-6_28.

[64] H. Haverinen, T. Janhunen, T. Päivärinta, S. Lempinen, S. Kaartinen, and S. Merilä, "Automating cybersecurity compliance in DevSecOps with open information model for security as code," in *Proc. 4th Eclipse Secur., AI, Archit. Model. Conf. Data Space*, Oct. 2024, pp. 93–102, doi: 10.1145/3685651.3686700.

[65] M. Voggenreiter, F. Angermeir, F. Moyón, U. Schöpp, and P. Bonvin, "Automated security findings management: A case study in industrial DevOps," in *Proc. 46th Int. Conf. Softw. Eng., Softw. Eng. Pract.*, 2024, pp. 312–322, doi: 10.1145/3639477.3639744.

[66] J. Garcia and A. Bartel, "Software security in DevOps," in *Proc. ACM Workshop Continuous Softw. Evol. Del. (CSED)*, 2016, pp. 20–26, doi: 10.1145/2896941.2896946.

[67] Z. Zhang, Y. Li, X. Zhu, and Y. Lin, "A method for modulation recognition based on entropy features and random forest," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. Companion (QRS-C)*, Jul. 2017, pp. 243–246, doi: 10.1109/QRS-C.2017.47.

[68] M. A. Ullah et al., "Review of techniques for integrating security in software development lifecycle," *J. Syst. Archit.*, 2025.

[69] T. Li, "Advancing software security and reliability in cloud platforms through AI-based anomaly detection," *J. Cloud Comput.*, vol. 13, no. 1, p. 45, 2024, doi: 10.1007/s13677-024-00345-6.

[70] R. N. Rajapakse, M. Zahedi, and M. A. Babar, "Collaborative application security testing for devsecops: An empirical analysis of challenges, best practices and tool support," 2022, *arXiv:2211.06953*.

[71] L. Prates and R. Pereira, "DevSecOps practices and tools," *Int. J. Inf. Secur.*, vol. 24, no. 1, Feb. 2025, Art. no. 11, doi: 10.1007/s10207-024-00914-z.

[72] M. Fu, J. Pasuksmit, and C. Tantithamthavorn, "AI for DevSecOps: A landscape and future opportunities," *ACM Trans. Softw. Eng. Methodology*, vol. 34, no. 4, pp. 1–61, May 2025, doi: 10.1145/3712190.

[73] T. R. Chura and Y. M. Kostiv, "Adaptation of information security in the agile world," *Comput. Syst. Netw.*, vol. 7, no. 1, pp. 307–312, Jun. 2025, doi: 10.23939/csn2025.01.307.

[74] *Guide for Conducting Risk Assessments*, National Institute of Standards and Technology (NIST), Gaithersburg, MD, USA, Sep. 2012.

[75] (2019). *Common Vulnerability Scoring System V3.1: Specification Document*. [Online]. Available: https://www.first.org/cvss/specification-document

[76] *Information Technology-security Techniques-Information Security Risk Management*, Standard ISO/IEC 27005:2022, International Organization for Standardization, Geneva, Switzerland, 2018, p. 2018.

[77] A. Gómez and J. C. Y. J. A. Mañas. (2012). *MAGERIT-Versión 3.0. Metodología De Análisis Y Gestión De Riesgos De Los Sistemas De Información, Libro I-Método, Madrid, España: Ministerio De Hacienda Y Administraciones Públicas, Centro Criptológico Nacional, Universidad Politécnica De Madrid*. [Online]. Available: http://administracionelectronica.gob.es/

ROBERTO CARLOS BAUTISTA RAMOS received the master's degree in information security from the International University of La Rioja and the master's degree in systems management from the Universidad de las Fuerzas Armadas (ESPE). He is currently pursuing the Ph.D. degree with the Escuela Politécnica Nacional.

He is a part-time Professor at International Technological University. He has collaborated as a member of Alpha Lab and Smart Lab: Research Laboratories in Cybersecurity, the IoT, and Smart Cities within the Faculty of Systems Engineering. His professional career has been focused on the design, implementation, and enhancement of robust, secure, and intelligent technological solutions, combining advanced technical knowledge with a strong ethical commitment and dedication to organizational development. He has led projects involving critical technology infrastructure, digital asset protection, process automation using artificial intelligence, and specialized technical training in both educational and corporate environments. Among his key strengths are honesty, responsibility, open-mindedness, diplomacy, analytical observation, and contextual awareness. He is versatile and adaptable, persistent in achieving objectives, with the ability to make well-informed decisions and act with ethical integrity even in complex situations. He works independently yet collaboratively, maintaining a continuous learning mindset, respecting cultural diversity, and promoting effective relationships in multidisciplinary teams.

SANG GUUN YOO (Senior Member, IEEE) received the Ph.D. degree (Hons.) (summa cum laude) from the Department of Computer Science and Engineering, Sogang University, Seoul, South Korea. He is currently a Professor with the Escuela Politécnica Nacional and a part-time Professor with the Universidad de las Fuerzas Armadas (ESPE). He is also a Professor of master's programs with PUCE, UTE, UCACUE, UTN, UTEG, and UNIR. He also collaborated as a Consultant for different entities, such as the Army Intelligence, the Ecuadorian Navy, the Ministry of Defense, and the Ministry of Tourism. He also had the opportunity to work as a Chief Research Engineer with LG Electronics, South Korea, and to collaborate on several academic-industry research projects with Samsung Electronics. He also directs the Smart Lab: Cybersecurity, the IoT, and Smart Cities Research Laboratory. He has also published 84 scientific articles indexed in international databases, such as WoS and Scopus. He is a Senior Member of international organizations, such as SCIEI.